

## ARE TWO HEADS BETTER THAN ONE FOR SOFTWARE DEVELOPMENT? THE PRODUCTIVITY PARADOX OF PAIR PROGRAMMING<sup>1</sup>

By: VenuGopal Balijepally  
College of Business  
Prairie View A&M University  
Prairie View, TX 77446  
U.S.A.  
vebalijepally@pvamu.edu

RadhaKanta Mahapatra  
College of Business Administration  
University of Texas at Arlington  
Arlington, TX 76019-0437  
U.S.A.  
mahapatra@uta.edu

Sridhar Nerur  
College of Business Administration  
University of Texas at Arlington  
Arlington, TX 76019-0437  
U.S.A.  
snerur@uta.edu

Kenneth H. Price  
College of Business Administration  
University of Texas at Arlington  
Arlington, TX 76019-0437  
U.S.A.  
price@uta.edu

### Abstract

*Extreme programming is currently gaining popularity as an alternate software development methodology. Pair programming, a core practice of this methodology, involves two programmers working collaboratively to develop software. This study examined the efficacy of pair programming by comparing the performance effectiveness and affective responses of collaborating pairs with those of individual programmers treated as nominal pairs. In a controlled laboratory experiment involving student subjects, proxies for entry level programmers working on entry level tasks, two factors were manipulated: programming setting (collaborative pair versus individuals) and programming task complexity (high versus low). Participants who worked in the individual condition were randomly combined into nominal pairs. The performance and affective responses of the collaborating pairs were then compared with those of the best performers and the second best performers of each nominal pair. Results indicated that programming pairs performed at the level above the second best performers and at the level of the best performers in each nominal pair. This relationship was found to be consistent across both levels of task complexity. Consequently, there was no evidence of an "assembly bonus effect," where the performance of a collaborating pair exceeds the performance of its best member working alone. While this finding may appear counterintuitive due to the general perception of two heads being better than one, it is consistent with the findings in small group research. When affective responses were considered, programming pairs reported higher levels of satisfaction than those of the best and second-best performing members in nominal pairs. They also showed higher levels of confidence in their performance*

<sup>1</sup>Carol Saunders was the accepting senior editor for this paper.

*compared to those of the second-best members. But the confidence levels of pairs were no different from those of the best performing members in nominal pairs. Theoretical and practical implications of these findings are presented.*

**Keywords:** Software development, agile methodology, pair programming, group problem solving, nominal group, team performance, assembly bonus effect

## Introduction

In the present knowledge economy driven by technological innovation, the ubiquity and growing importance of software products and services are all too evident. Despite some fluctuations due to business cycles, organizations are continuing to invest in software products and services to stay afloat in a hyper-competitive business environment. For instance, sales of software in the United States grossed \$217.7 billion for 2007 (excluding embedded and bundled software), registering a healthy 7.1 percent growth over the previous year.<sup>2</sup> Based on Forrester Research projections, software purchases globally were also expected to grow similarly at 7 percent for 2007 (ITPro 2007).

Despite the tremendous strides made in software development, as evidenced by the rich array of methods, tools, and techniques, about 19 percent of all software projects are never completed, while another 46 percent are categorized as “challenged.” These challenged projects are operational, but over-budget, over the time-estimate, and completed with fewer features and functions than originally specified (Rubinstein 2007). The growing frustration spurred by the dismal success rates of software projects, coupled with the imperative for responsiveness and agility, has ushered in an array of development methods that differ appreciably from traditional software practices. Several software luminaries, who were seeking alternatives to the traditional plan-driven software development, came together in 2001 to draft the Agile manifesto,<sup>3</sup> heralding agility in software development. These new methods, labeled *Agile Development Methodologies*, aim to expeditiously deliver software of high-quality and value to customers by emphasizing the following: (1) collaborative and empowered teams unfettered by rigorous processes;

(2) simplicity of design and minimal critical specifications, while documenting only what is absolutely necessary; (3) active involvement of preferably colocated customers; and (4) inevitability of change and an understanding that it may be leveraged through rapid iterations, feedback, and constant reflection on the consequences of actions (Cockburn 2002; Cockburn and Highsmith 2001; Highsmith and Cockburn 2001). A singularly distinctive feature of these methods is the premium they place on collaborating and self-organizing teams (Cockburn 2002). SCRUM, eXtreme Programming (XP), Dynamic Software Development, Adaptive Software Development, Crystal Methods, and Feature-Driven Development are some of the more popular agile methods. The growing popularity of agile methodologies can be gauged from a recent survey of software developers, where 69 percent of respondents indicated that their organizations are using agile methods and another 7.3 percent hinted that they should be going agile in the next year (Ambler 2007).

Among the agile methodologies, eXtreme Programming (XP), is by far the best documented and the most popular (Orr 2002). XP is a collection of proven and not-so-proven practices of software development, combined into a coherent methodology. Proponents of XP insist on implementing all of the 12 core practices together, as they form a coherent system, compensating for the lack of significant up-front design and documentation (Beck 1999). Despite gaining increasing favor among software developers, empirical research that sheds light on the efficacy of the core principles underlying such methods is still evolving.

XP stipulates pair programming as a core practice, where two programmers, sharing the same computer, work collaboratively on all aspects of software development (Williams and Kessler 2000). This practice of programming in pairs has attracted some research interest in the academic community. The oft-cited works of Nosek (1998) and Williams (2000) suggest that programming pairs not only outperform individual programmers on quality, but also produce fewer defects, are more confident of their solutions, and derive greater satisfaction from the problem solving/programming activity. Other benefits to pairing reported by Williams included enhanced learning, higher problem solving skills, and improved team building. Pairing programmers of varying technical abilities is considered particularly beneficial for cross training and redistribution of technical expertise within project groups (Fruhling and de Vreede 2006).

Several other studies have also attested to benefits of pair programming, such as enhanced software quality in large-scale software projects, reduced code defects, better designs, and enhanced predictability in development time and program size

<sup>2</sup>“National Economic Accounts: GDP Expanded Detail Table,” Bureau of Economic Analysis, August 2008 (<http://www.bea.gov/national/pdf/dpga.pdf>).

<sup>3</sup>“Manifesto for Agile Software Development,” Agile Alliance, 2001 (<http://www.agilemanifesto.org>).

(Canfora et al. 2007; Cao et al. 2004; Nawrocki and Wojciechowski 2001; Phongpaibul and Boehm 2006; Xu and Rajlich 2006). A recent meta-analysis of studies comparing the effectiveness of pair programming over individual programming reported moderate benefits in terms of quality and reduction in time to completion (Dybå et al. 2007). That study further reported a moderate increase in effort (person-hours) when developers pair up. Another recent study also attests to the benefits of pair programming and strongly advocates it as a pedagogical tool in programming courses (McDowell et al. 2006).

Some other studies, however, claim that pair programming is less efficient than reported by previous studies (e.g., Arisholm et al. 2007; Nawrocki and Wojciechowski 2001). These studies could neither find pair superiority in software quality (Arisholm et al. 2007; Heiberg et al. 2003; Madeyski 2006) nor uncover reduction in code defects (Hulkko and Abrahamsson 2005; Nawrocki and Wojciechowski 2001; Vanhanen and Lassenius 2005). Table 1 summarizes the findings from previous studies on the effectiveness of pair programming over individual programming. Consistent with prior literature (e.g., Nosek 1998; Williams et al. 2000), we use the terms pair programming and collaborative programming interchangeably in this paper.

Despite these efforts to illuminate the question of whether pairs outperform individuals in programming tasks, the findings may be considered tentative at best, due to differences in research design and methodological rigor across these studies. While experimental controls, randomization and research rigor are typically consistent across studies in other disciplines such as social psychology, variations across these factors in software development research make it difficult to compare the findings. For instance, some pair programming studies used student homework assignments as experimental tasks, where laboratory controls would not be feasible (e.g., Canfora et al. 2005; Phongpaibul and Boehm 2006; Vanhanen and Lassenius 2005; Williams 2000). In addition, some have used repeated measure designs, where same subjects worked individually and in pairs (e.g., Canfora et al. 2007; Canfora et al. 2005; Müller 2005), while others have employed quasi-experimental designs for the assignment of subjects to treatments (e.g., Arisholm et al. 2007). The approach to partner selection also differed considerably, ranging from self-selection by pairs (Canfora et al. 2005; Williams 2000) to matching pairs on abilities such as GPA or experience levels (Nawrocki and Wojciechowski 2001; Phongpaibul and Boehm 2006). Yet another source of variation is the unit of analysis employed in these studies. For example, certain studies used the programming team as the unit of analysis (e.g., Baheti et al. 2002; Heiberg et al. 2003;

Phongpaibul and Boehm 2006; Vanhanen and Lassenius 2005), whereas others used a dyad.

There is, therefore, a dearth of *controlled* experimental studies that seek to establish the efficacy of this important practice. A review of the extant literature on pair programming also shows scant attention to behavioral and social theories that have the potential to reveal factors paramount to the success of pairs. While a few studies have drawn on theoretical insights from cognitive perspectives, for example, the theory of human problem solving (Newell and Simon 1972) and distributed cognition theory (Flor and Hutchins 1991) or from Maslow's (1943) theory of individual needs and motivation, none, to the best of our knowledge, has considered the spectrum of factors that behavioral theories deem necessary for an adequate explanation of group, and by extension, paired-programming performance.

Additionally, current studies on pair programming are not making the most rigorous comparison when contrasting paired with individual programmers. These studies compare the performance of a programming pair with that of an average individual. A pair could be expected to outperform an individual in problem solving based on pure probabilities alone, as there is a higher chance for a pair to have a competent programmer who could code the solution. A more rigorous and practically relevant comparison would be between a collaborating pair and the best and second best members of a nominal pair. Nominal pairs are randomly created pairs of individuals that have worked alone, but whose efforts are conjointly considered. These pseudo-pairs allow researchers to directly assess the potentially positive impact of interaction on performance when individuals work collaboratively, using an equal man-hour comparison. When the performance of a dyad or larger group exceeds the performance of its best member, it is referred to as *assembly bonus effect* in the social psychology literature (Collins and Guetzkow 1964). In light of prior evidence suggestive of programming pair superiority over average individuals in achieving higher software quality (Nosek 1998; Williams 2000), it would be interesting to see how software quality achieved by programming pairs would measure up to this new benchmark.

In summary, given the increasingly social nature of software approaches, it is critical to have a good grasp of the factors that affect group performance in a software development context. With increasing acceptance and popularity of XP and other agile methodologies, there is a need to investigate the efficacy of the core practices such as pair programming, which have large cost and productivity implications for the software development community. While there is some evidence suggestive of higher satisfaction (Nosek 1998; Wil-

**Table 1. Research Findings on the Effectiveness of Pair Programming**

| Positive Findings                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Negative/Neutral Findings                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Software Quality</b> <ul style="list-style-type: none"> <li>Enhanced quality (Nosek 1998; Williams 2000)</li> <li>Enhanced quality in large software projects (Cao et al. 2004)</li> <li>Enhanced quality of software designs (Canfora et al. 2007; Williams 2000)</li> <li>Reduced defects (Phongpaibul and Boehm 2006)</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                               | <ul style="list-style-type: none"> <li>No differences in the package level quality metrics (Madeyski 2006)</li> <li>No differences in proportion of correct solutions (Arisholm et al. 2007)</li> <li>No differences in percentage of correctly implemented test cases in a given time (Heiberg et al. 2003)</li> <li>Higher defects in pair solutions (Vanhanen and Lassenius 2005)</li> <li>No differences in errors uncovered in acceptance tests (Nawrocki and Wojciechowski 2001)</li> <li>No differences in defect density (Hulkko and Abrahamsson 2005)</li> <li>No effect on the thoroughness or effectiveness of testing when using test driven development (Madeyski 2007)</li> </ul> |
| <b>Development Effort</b> <ul style="list-style-type: none"> <li>Effort comparable if required to produce programs of similar level of correctness (Müller 2005, 2006)</li> <li>No productivity differences (Hulkko and Abrahamsson 2005)</li> <li>Reduced development effort in student programmer group (Phongpaibul and Boehm 2006)</li> <li>Reduced effort (time spent) (Canfora et al. 2005)</li> </ul>                                                                                                                                                                                                                                                                                                                         | <ul style="list-style-type: none"> <li>Required more effort to perform the task correctly (Arisholm et al. 2007)</li> <li>Pair lower in productivity (sum of implemented use cases divided by effort) (Vanhanen and Lassenius 2005)</li> <li>Required more development effort in professional programmer group (Phongpaibul and Boehm 2006)</li> <li>No differences in time to completion (Arisholm et al. 2007; Nawrocki and Wojciechowski 2001; Rostaher and Hericko 2002)</li> <li>Pair programming as in XP less efficient (Nawrocki and Wojciechowski 2001)</li> </ul>                                                                                                                     |
| <b>Task Complexity</b> <ul style="list-style-type: none"> <li>Reduced time to completion on low complexity tasks for experienced programmers (Arisholm et al. 2007)</li> <li>Flexible pair programming beneficial for large scale projects with complex software development (Cao et al. 2004)</li> <li>Increased correctness of solution on complex tasks for junior programmers (Arisholm et al. 2007)</li> <li>Most useful for learning and complex tasks (Hulkko and Abrahamsson 2005)</li> </ul>                                                                                                                                                                                                                                | <ul style="list-style-type: none"> <li>No reduction in time taken to solve more complex tasks correctly (Arisholm et al. 2007)</li> <li>No effect of task complexity on the effort between pairs and individuals (Vanhanen and Lassenius 2005)</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Others</b> <ul style="list-style-type: none"> <li>Higher adherence to coding standards due to peer pressure (Cockburn and Williams 2001)</li> <li>Higher enjoyment with problem solving process (Nosek 1998; Vanhanen and Lassenius 2005; Williams 2000)</li> <li>Higher confidence in solutions (Nosek 1998; Williams et al. 2000)</li> <li>Enhanced learning due to cross training (Williams 2000; Williams et al. 2000)</li> <li>Improved team building (Williams 2000)</li> <li>Collaborative and supportive environment fostered (Cao et al. 2004)</li> <li>Higher problem solving skills (Williams 2000)</li> <li>Enhanced predictability of development time and program size (Nawrocki and Wojciechowski 2001)</li> </ul> | <ul style="list-style-type: none"> <li>Lower adherence to coding standards but has higher comment ratio (Hulkko and Abrahamsson 2005)</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

liams 2000) and higher confidence in solution (Nosek 1998) among programming pairs compared to average individual developers, it remains to be seen whether pairs would be more satisfied and more confident relative to the best and second-best members of nominal pairs.

The primary objectives of this study are (1) to examine the efficacy of pairs vis-à-vis individuals in programming tasks; (2) to study the effect of pairing on affective reactions of programmers, including their satisfaction and confidence in performance; and (3) to explore how programming task complexity impacts performance and affective reactions. In examining these questions, we incorporate research findings and a methodological framework from group performance studies (Laughlin and Ellis 1986; McGrath 1984; Steiner 1972), a literature that has a long standing and respected tradition in social psychology. Thus, this research, while retaining a cognitive orientation consistent with MIS studies (e.g., Flor and Hutchins 1991), makes the following contributions:

1. It introduces behavioral theories and perspectives on individual versus group problem solving to theoretically guide and explore the phenomenon of pair programming. Specifically, in this study, we draw from theories on social facilitation, social loafing, individual and group information processing, and the rich body of empirical evidence from social psychology.
2. Employing a procedure consistent with recent research designs on group performance in social psychology (e.g., Laughlin, Zander et al. 2003), it implements a rigorous comparison between pair performance and the performance of the best and second best individuals working in nominal pairs. This mode of comparison allows us to shed light on an enduring research question in group performance, the *assembly bonus effect*. This effect, which enables a group to perform above the level of its best member working alone, is yet to be studied in the context of pair programming.
3. Using a multivariate approach, it examines affective outcomes (participant *satisfaction* and *confidence in performance*) of pairs and individuals to complement *software quality*.
4. It examines the contingent effect of task complexity on the relative performance effectiveness of programming pairs and individuals working in nominal pairs.

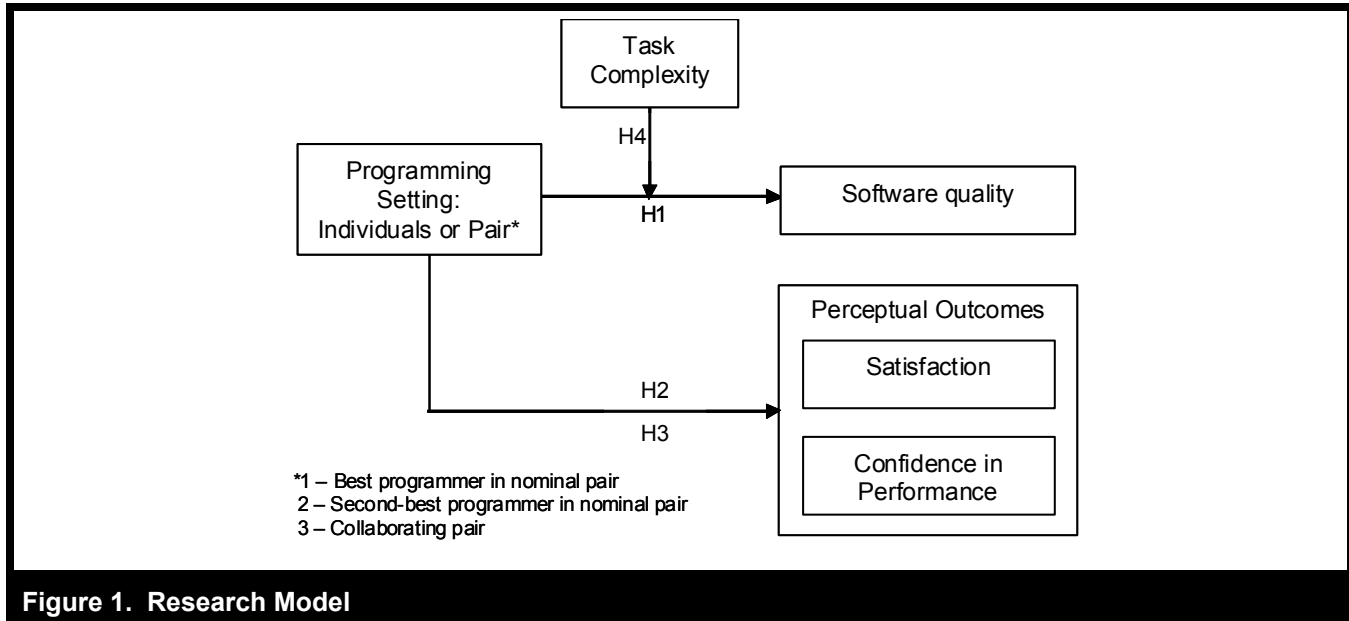
The remaining sections of the paper are organized as follows. First, an overview of the theoretical literature that forms the

framework for the research questions is provided. Second, the hypotheses of the study are described. Third, the research methodology is outlined, followed by a description of the results. Fourth, the results are discussed along with implications for practice and future research directions. Finally, conclusions are drawn.

## Research Framework

Our research questions focus on the relative efficacy of pair versus individual programming on performance (*software quality*) and the moderating effect of *task complexity* on this relationship. Additionally, we examine the relative efficacy of pair versus individual programming on developers' affective responses to the task (i.e., participant *satisfaction* and *confidence in performance*). Figure 1 showcases the overall research model of this study.

In this report, we use the terms *pair*, *dyad*, and *group* interchangeably, depending on the literature being examined. Pair has its origins in the IS literature (Beck 2000; Williams and Kessler 2000), while dyads and groups are more commonly found in the sociology/social psychological literatures (e.g., Nixon 1979). More importantly, however, we treat these entities as similar, based on their properties. There are differences between pairs (dyads) and groups, which include voting to arrive at decisions, the possibility of coalition formation, majority and minority opinions splits and influence attempts, and the impact of the loss of a group member (Simmel 1950). However, pairs (dyads) and groups of larger size are also similar in many ways, including task interdependence, the need to agree/produce a single outcome to represent the interests of the members, the importance of mutual interaction and influence in making decisions and solving problems, the possibility of a shared identity, and a role structure characterizing status and work differences among members. These facets are often used to define boundary conditions differentiating groups from individuals, and theorists consider these facets as characteristics of "groups" of size two or larger (see Forsyth 1999; McGrath 1984). Thus, pairs or dyads are considered to be similar to larger size groups, based on broad similarities in social behavior (Levine and Moreland 1998). As such, research on task groups is appropriate to provide a guiding framework for the current research on programming pairs or dyads. Additionally, a methodology emphasizing nominal group performance as a baseline index to evaluate the impact of programming pairs is context free and appropriate, irrespective of the size of the programming group that is being considered.



**Table 2. Important Task Attributes Defined in Group Task Typologies**

| Task Attribute       | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unitary              | Task cannot be divided among group members (Steiner 1972); for example, tug of war.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Divisible            | Task can be divided into subtasks, each of which can be performed by a different member of the group (Steiner 1972); for example, painting a house.                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Maximizing           | Emphasis on quantity of group performance (Steiner 1972); for example, generating several ideas.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Optimizing           | Emphasis on quality of group performance (Steiner 1972); for example, generating the best idea.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Disjunctive          | Requires only one member of the group to perform the task for the task to be completed. The potential of the group is defined by its best performing member (Steiner 1972); for example, puzzles with yes/no or either/or solutions.                                                                                                                                                                                                                                                                                                                                                         |
| Conjunctive          | Requires each group member to contribute for the task to be completed. Group performance is limited by its worst performing member (Steiner 1972); for example, product assembly line.                                                                                                                                                                                                                                                                                                                                                                                                       |
| Judgmental           | Task with no correct answers. Group response is defined by the consensus evolved over the evaluative, behavioral, or aesthetic preferences of the group members (Laughlin 1980). Also referred to as decision-making tasks (McGrath 1984); for example, jury task.                                                                                                                                                                                                                                                                                                                           |
| Intellective         | Tasks with intuitively compelling or demonstrably correct answers based on a verbal or quantitative conceptual system (Laughlin 1980); for example, math, analogy and word problems.                                                                                                                                                                                                                                                                                                                                                                                                         |
| Task Demonstrability | <p>The ease with which the correct answer could be verified.</p> <p>Conditions determining task demonstrability (Laughlin and Ellis 1986):</p> <ul style="list-style-type: none"> <li>• Group evolves consensus on a conceptual system defining the problem.</li> <li>• Group has sufficient information to solve the problem.</li> <li>• Group member suggesting the correct response has sufficient ability, motivation and time to demonstrate it to other members.</li> <li>• Other group members are able to recognize the correct response once proposed by any one member.</li> </ul> |

## Framing Programming Tasks in Terms of Group Task Typologies

Task type has been central to understanding the contingent nature of small group performance. Early group task typologies (e.g., Laughlin 1980; McGrath 1984; Steiner 1972) have been invaluable in explaining how task characteristics influence group performance. Table 2 summarizes important task attributes from these task typologies that are relevant to the programming context. Following Steiner's (1972) task typology, programming tasks have characteristics of disjunctive tasks, which require one group member to perform the task for the task to be completed. Additionally, programming tasks are typically *unitary* (not easily divisible among group members) and *optimizing* (quality of output rather than speed is emphasized). Therefore, even if one member of the pair can solve the problem or perform the task, the pair has the potential to successfully complete the task.

Programming tasks also share some characteristics with tasks that are labeled as intellectual tasks (Laughlin 1980). These are essentially disjunctive tasks, but with demonstrably correct answers. Examples include mathematical, vocabulary, and analogy problems. As with intellectual tasks, programming tasks may vary in the ease with which the correct answers can be verified. When solutions are easily verifiable, recommendations proposed by one member may be quickly adopted by the other. Increase in solution verifiability is likely to reduce potential decision conflict and enhance the accuracy of the proposed solution.

## Forces Affecting Dyad Performance

According to the distributed cognition framework (Flor and Hutchins 1991), an information processing perspective, programmers are likely to accrue performance benefits when they work together. Through interaction, dyad members are able to search through a larger space of alternatives, sharing memory for old plans, while jointly producing ambiguous code segments with fewer defects (Flor and Hutchins 1991). Active communication, involving perspective taking and perspective making (Boland and Tenkasi 1995), is an essential part of this process. This facilitates the complex cognitive processing necessary to achieve quality solutions. If one views programming as completing a large set of interrelated disjunctive tasks, pairs can benefit from a large resource pool. For example, group problem solving can be facilitated by individual members who hold different and unique sets of information (Hinsz et al. 1997).

When working on intellectual problem solving tasks with large information processing requirements, groups typically outperform average individuals (Hill 1982), with group performance approaching that of its most capable member, or even exceeding it in some rare cases (e.g., Laughlin, Zander et al. 2003). Demonstrability of task is critical to group problem solving performance. Laughlin and colleagues argue that if sufficient information is available within a group, problem solving is facilitated when: (1) there is a group consensus on the conceptual system needed to solve the problem; and (2) group members have the time, ability, and motivation to demonstrate the correct response to other members. This also entails that group members are able to recognize the correct solution once it is proposed by any member in the group (Laughlin and Ellis 1986). Task demonstrability is also defined in Table 2 for easy reference.

We argue that programming tasks have moderate demonstrability. When pairs are asked to work collaboratively on a programming task, they are expected to have sufficient information concerning the problem definition, but need to explore class libraries and API (application programming interface) documentation for technical guidance during code development. The partners could typically arrive at a consensus on a conceptual system leading to the solution. However, if programming pairs are to perform at their maximal levels, it is critical that members have the ability to identify and utilize the competencies of their partners (Bonner 2004; Bonner et al. 2002). While no explicit competency information is typically provided, the process of coding, compiling, and debugging, inherent in programming tasks, should help dyad members get a sense of the relative competencies of their partner. In addition, XP practices require the pair to interchangeably play the roles of driver and navigator during code development. This will further enable each member to assess the relative strengths of the partner. This process of role interchange also facilitates perspective taking and perspective making, thus leading to more systematic integration of member skills. Additionally, when and if program compilation provides quick proof of a "truth-wins" approach (a social combination process where a member with the correct solution is necessary and sufficient for the correct group response), one could expect that solution to be adopted (Laughlin and Ellis 1986).

However, there may also be elements of programming that represent what McGrath (1984) labels as decision-making tasks. For example, deciding whether efficiency versus maintainability considerations of programming algorithms and structures are more critical may not be based on facts, but on

the values, beliefs, and attitudes about the merits of alternate solutions. When these types of judgments are involved, moving forward requires a consensus. Thus, we argue that programming tasks are not pure intellectual tasks, but involve some decision-making component. Consequently, we consider the overall demonstrability of programming tasks to be moderate and not as high as found in pure intellectual tasks, where solutions are easily verifiable.

There are other processes that reflect motivation, coordination, and cognitive concerns that can influence the level of group performance achieved. Social loafing and social facilitation are two such processes that potentially inhibit pair performance.

Social loafing occurs when individuals working together in groups exert less effort than when they work individually (Karau and Williams 1993). Factors that promote social loafing include a perception that one's efforts are dispensable (Harkins and Petty 1982), a reduced responsibility for the final outcome (Petty et al. 1977), and the inability to identify an individual's contributions to the group's effort (Williams, Harkins and Latane 1981). We would argue, however, that productivity losses due to social loafing in pair programming may be minimal. Considering the fact that the programming task is highly meaningful to programmers, we expect loafing to be minimized (Brickner et al. 1986). Additionally, programming procedures stipulated in XP may also discourage loafing (e.g., pairs taking turns at the keyboard). When working especially on nonroutine task components, which are a substantial part of most programming tasks, dyad members experience less redundant effort and, therefore, less feeling of dispensability of their effort (Harkins and Petty 1982). We expect that while working in dyads, rather than larger groups, dyad members are more likely to be continually aware of the effort put in by each other, thus making their contribution more identifiable, thereby increasing personal responsibility for the final outcome (Latane et al. 1979). Therefore, we anticipate social loafing effects to be minimal when XP programming procedures are used.

Research on social facilitation has reported that in the presence of another observant individual, performance in well-learned tasks is enabled, while performance in novel or more complex tasks is hampered (Aiello and Douthitt 2001; Harkins 1987; Zajonc 1965). Since the observer could be evaluating the performance, or simply be a source of cognitive distraction, working in the presence of a partner could hinder performance. In pair programming, however, the partner is an equal stakeholder in the problem solving activity, actively collaborating and engaging in the problem situation (Williams and Kessler 2000). Consequently, we do not expect loss of performance due to the presence of a partner.

## **Performance Expectation in Pair Programming**

The preceding discussion indicates that multiple factors affect the performance of programming pairs. One way to evaluate the performance of a programming pair is to compare it against those of its members working alone. An assembly bonus effect is said to occur when group performance is greater than the performance any of its members could accomplish alone, or by a combination of individual efforts (Collins and Guetzkow 1964). This is also referred to as *synergy*, in work team effectiveness (Hackman 1987) and brainstorming (Dennis and Valacich 1993) literatures. In a disjunctive task, such as programming, an assembly bonus effect is realized when the performance of a pair exceeds that of its most competent member. Other standards of comparison include whether the programming pair will perform at or below the level of its most competent member (Laughlin, Bonner and Altermatt 1998; Laughlin, VanderStoep and Hollingshead 1991). When considering performance below the most competent member of the pair, more fine grained comparisons could include whether performance is at or above the level of the least competent member of the group (Laughlin, Bonner and Altermatt 1998; Laughlin, VanderStoep and Hollingshead 1991). Research has demonstrated that performance above the most competent member of the group is quite rare, as groups may have difficulty avoiding certain process losses inherent in collaborative work (Hill 1982). Small group research scholars caution that such effects, when found, are usually modest, and every such claim needs careful evaluation, as it is easy to underrate group potential and thereby overstate group performance (Kerr and Tindale 2004). Studies that have demonstrated performance above the most competent member of the group on intellectual tasks, the type of task we most closely associate with programming tasks, used groups consisting of three or more members (e.g., Laughlin, Bonner and Miner 2002; Laughlin, Zander et al. 2003). To our knowledge, there is no empirical evidence of performance above the best member when problem solving dyads are considered. Hence, we do not expect this to occur in pair programming. However, empirical evidence from small group research also documents that group performance typically exceeds the performance of the least competent member (Laughlin, Bonner and Altermatt 1998; Laughlin, VanderStoep and Hollingshead 1991). Consequently, based on these findings, we would expect pair performance to exceed that of the second best (least competent) member, but not the best member of a nominal pair. Therefore, our first hypothesis is

*Hypothesis 1: While working on a task using XP procedures, programming performance of a collaborating pair measured in terms of software quality*

*will be higher than the performance of the second-best member of a nominal pair.*

### **Pair Programming Satisfaction and Confidence**

Research findings suggest that group work, in general, is more enjoyable compared to working alone (Garibaldi 1979). Researchers argue that, in part, this is a consequence of humans being a “group-seeking species” (Forsyth 1999). Group membership fulfills multiple social and emotional human needs (Levine and Moreland 1998). There is also considerable evidence that working in groups has a positive impact on factors related to perceptions of performance. Compared to individuals working alone, group members tend to have higher goal commitment, more positive attitude toward goal attainment, and report higher *satisfaction* with their performance (Hinsz and Nickell 2004). Group members tend to be more satisfied with their performance, even when their performance does not differ from that of individuals (Hinsz 1995). Similarly, research on brainstorming suggests that perceptions of performance are higher when group brainstorming is contrasted with individual brainstorming (Paulus et al. 1993). Also, on problem solving tasks, group members tend to report higher confidence in their solutions (Snizek 1992; Stephenson and Wagner 1989).

In the IS literature, higher levels of *satisfaction* have also been reported when pair programming was compared with individual programming (Nosek 1998; Williams 2000; Williams et al. 2000). In these studies, indexes measuring perceptions of satisfaction and performance were aggregate measures, when participants worked in groups. These mean scores were then compared with the average scores of individuals working alone. Although mean levels of confidence and satisfaction were higher in pairs compared to those of average individuals, these levels may not be uniformly higher when compared to those of individual members of nominal pairs. However, consistent with the trend in the literature, we predict higher levels of *satisfaction* and *confidence in performance*, when pairs are contrasted with individuals working alone. Specifically, we expect the satisfaction and confidence levels among pairs to be higher than those of the best and the second-best members of the independently working nominal pairs. Therefore, our next hypotheses state

*Hypothesis 2A: In a collaborating pair working with XP procedures, the mean satisfaction with the programming task will exceed the level reported by the best member of a nominal pair.*

*Hypothesis 2B: In a collaborating pair working with XP procedures, the mean satisfaction with the programming task will exceed the level reported by the second-best member of a nominal pair.*

*Hypothesis 3A: In a collaborating pair working with XP procedures, the mean confidence in performance will exceed the level reported by the best member of a nominal pair.*

*Hypothesis 3B: In a collaborating pair working with XP procedures, the mean confidence in performance will exceed the level reported by the second-best member of a nominal pair.*

### **Moderating Effect of Task Complexity**

Complexity of a task emanates from an increase in information load, information diversity, and rate of information change (Campbell 1988). This can potentially influence the process losses and process gains when individuals work collaboratively on tasks of differing levels of complexity. On the inhibitory side, task demonstrability decreases with increasing complexity, thus making the correct solution less obvious, even when identified by one of the dyad members. Thus, in discussing the merits of alternate solutions, the communication requirements increase dramatically, placing extra demands on the time and effort of the dyad. The member with the correct solution could easily get overruled by the partner and the dyad could end up exploring wrong leads and dead ends. We, however, argue that potential process gains of collaborative working outweigh these process losses.

According to human information processing literature, with increasing task complexity, there is a corresponding increase in the complexity of information needed to solve the problem. Consequently, there is an increase in the number of information sources accessed (Bystrom and Jarvelin 1995). The performance of an independently working individual increases with increasing task complexity when the task demands are lower than the cognitive capacity of the individual, beyond which it starts to deteriorate (Schroder et al. 1967). In groups, higher levels of task complexity are associated with higher levels of participation by members in decision making and boundary spanning activities (Ito and Peterson 1986). So, programming pairs could respond to increasing task demands with increased collaboration and intense information seeking activities. Pairs, with their ability to process larger amounts of information relative to independently working individuals (Hinsz et al. 1997), are thus likely to be more effective as task complexity increases (Laughlin, Bonner and Altermatt 1998).

Also, as we have argued earlier, the XP process is designed to facilitate pair performance through role assignment (such as driver and navigator) and role swapping between partners. In addition, the distributed cognition view attributes several benefits to pairing, including sharing of goals and plans, searching through larger space of alternatives, and joint production of ambiguous problem components (Flor and Hutchins 1991). These are particularly helpful when a pair works on more complex tasks. Such benefits are maximized when there is division of labor in a collaborative interaction system with efficient communication. For instance, a programming pair implementing some new application programming interfaces (APIs) or interfacing with legacy applications could benefit immensely if the navigating partner is able to look up the documentation and constantly feed options to the implementing partner. Advocates of pair programming have long speculated that the performance benefits of pairing should be higher for more complex tasks due to greater information processing capacity in pairs (Williams et al. 2000). Consequently, we expect collaborating pairs to be less affected by task complexity than independently working members of nominal pairs. Therefore, our next hypothesis is

*Hypothesis 4: Task complexity will more adversely influence the performance of the best person in a nominal pair than the performance of a collaborating pair.*

## Method

### Experimental Design

A laboratory experiment, employing a  $2 \times 2$  factorial design, was conducted to examine the effects of different modes of programming (individual versus pair) and task complexity (low versus high) on outcomes of *software quality*, *satisfaction*, and *confidence in performance*. The experiment simulated a software development task. In the individual programming condition, participants worked independently, while in the paired programming condition, participants worked interdependently on the programming tasks. A total of 122 subjects participated in the study. They were randomly assigned to the pair condition or the individual condition, resulting in 30 dyads and 62 individuals. Two of the subjects assigned to the individual condition were subsequently dropped from the analysis because of incomplete responses. The remaining 60 subjects, who worked independently, were randomly combined into 30 nominal pairs for statistical analysis. Consequently, 30 collaborating pairs and 30 nominal pairs were involved in the research design. The best (most competent) and the second best (least competent)

members of each nominal pair were identified based on their actual performance on the experimental task. The second factor of interest was task complexity, which was varied at two levels (high versus low). The task of low complexity involved modifying five methods in two classes, while the more complex task involved modifying seven methods in five classes. The difference between the complexities of the two experimental tasks, therefore, emanated primarily from the multiplicity of solution paths (Campbell 1988). The two programming tasks designed and used for this purpose are described in Appendixes A and B.

For the purpose of analysis, the combination of programming setting and task complexity resulted in a  $3$  (programming setting: collaborating pairs versus best nominal group members versus second best nominal group members)  $\times 2$  (high task complexity versus low task complexity) factorial design. There were 15 replications in each of the six cells of the design.

### Participants

The participants were drawn from undergraduate and graduate students enrolled in Information Systems courses at a large public university in the United States. Knowledge of the Java programming language was a requirement for participating in the experiment. Inclusion of graduate along with undergraduate students was partly dictated by logistical considerations, as the available subject pool with programming skills in Java was relatively small. We, however, viewed this as a positive factor because it resulted in a pool of participants with diverse skills and abilities. Participation in the study was voluntary. Subjects earned class credit for their participation. Subjects were also included in a lottery to win one of three prizes, each of \$50 value. To motivate them further to perform well, they were informed that an amount in cents, equal to their performance scores, would be contributed by the experimenter to a charity. Subjects who were unwilling to participate, or had participated in a previous semester, were provided with alternative home assignments by their respective instructors.

The mean (standard deviation) age of the experimental subjects was 27.50 (6.39) years. The experiment was conducted over three semesters and the distribution of subjects across the three semesters was as follows: Spring – 49 (40.8 percent), Summer – 25 (20.8 percent), and Fall – 46 (38.3 percent). In each semester, participants were randomly assigned to treatments to make the number of participants nearly equal in each of the four treatment conditions. The demographic characteristics of the experimental subjects are described in Table 3.

**Table 3. Demographic Details of Experimental Subjects**

| Demographic Variable   | Number of Subjects | Percentage |
|------------------------|--------------------|------------|
| Gender                 |                    |            |
| Male                   | 89                 | 74.2       |
| Female                 | 31                 | 25.8       |
| Subject status         |                    |            |
| Undergraduate          | 99                 | 82.5       |
| Graduate               | 21                 | 17.5       |
| Subject Citizenship    |                    |            |
| U.S.                   | 81                 | 69.2       |
| Other countries        | 36                 | 30.8       |
| Programming experience |                    |            |
| 0 – 1 year             | 44                 | 37.9       |
| 1 – 2 years            | 30                 | 25.9       |
| 2 – 4 years            | 20                 | 17.2       |
| > 4 years              | 22                 | 19.0       |
| Java experience        |                    |            |
| 0 – 1 year             | 85                 | 73.3       |
| 1 – 2 years            | 24                 | 20.7       |
| 2 – 4 years            | 6                  | 5.1        |
| > 4 years              | 1                  | 0.9        |

### **Experimental Setting, Procedures, and Manipulations**

The experimental setting was the research lab of the college of business. All eight cubicles in the laboratory were insulated and contained laptop computers loaded with Java JDK5, Java documentation, and Notepad, to simulate the Java development environment. The laptops were standalone, hence, subjects had no access to resources on the Internet.

After being seated in the experimental laboratory, participants were informed that they would be working on a programming task involving application development in Java. The subjects' participation involved working on a warm up task for 15 minutes and on an experimental task for 2 hours. The warm up task was used for familiarizing subjects with the computers and the experimental setting. For the programming pairs, the warm up task also afforded the opportunity to practice working collaboratively. The main experimental task lasted for 2 hours, which was determined based on feedback provided by subjects during the pilot test.

Stopwatches were used to time the beginning and end of the warm-up task and the main experimental task. This was to ensure that participants worked for the same amount of time

on the experimental task. If participants were not able to complete the task during the assigned time of 2 hours, their work was still collected and performance assessed. After completing the experimental task, each subject filled out a questionnaire, which was designed to collect responses to some of the dependent measures, manipulation check questions, and demographic data items. Each experimental session lasted for a total of 3 hours.

### **Pilot Test**

Prior to the main experiment, a pilot test was conducted to test the experimental protocols of all four treatment conditions. At the end of the pilot, participants' judgments on the lab setting, programming environment, experimental task, and adequacy of time were elicited. Based on this feedback, appropriate changes were made to the scripts, and duration of sessions, as well as to the logistics of the experiment.

### **Manipulation of Independent Variables**

For treatment groups involved in pair programming, written instructions were provided on how they should work collabo-

ratively, as stipulated in XP (Williams et al. 2000). These instructions included how one of them would be working as the driver at the keyboard, while the partner would play the role of the navigator. Instructions also described the transferring of the keyboard to the partner, and switching roles of driver and navigator at frequent intervals. To encourage active communication between partners, they were informed that the cubicles were insulated and that they could converse freely without any fear of disturbing others. To ensure that the procedures were enacted and collaborating pairs switched roles, the experimenter visited collaborating pairs every 15 minutes to remind them of these requirements. A log of such visits was maintained to help verify experimenter's compliance with this procedure. In the individual programming conditions, participants worked alone in cubicles. The experimenter visited with these subjects at roughly 30 minute intervals to ensure that instructions were being followed. Short breaks were provided to all participants in a staggered manner, as needed, to prevent interaction among participants from across treatments.

## Dependent Variables

Three dependent variables were measured: *software quality* (SQ), *satisfaction* (S), and *confidence in performance* (CP).

*Software quality* represents the assessment of the quality of the task performed by the individual, or the pair, on the programming task. It is reflective of how well the code satisfied the requirements stipulated in the problem statement and produced correct results. To rate the solution qualities of the two experimental tasks, two separate assessment rubrics were designed by one of the authors. The solutions were evaluated on a scale of 0 to 125. Points were awarded based on correctness of constructors, methods and their parameters, code generalizability, comment statements, code indentation etc. Two Information Systems doctoral students, not directly connected with the study, were trained as raters. The assessment rubrics were further refined based on their experience in scoring software quality during the pilot test. Copies of the software quality assessment rubrics can be found in Appendixes C and D. For the main experimental tasks, the two raters independently evaluated the code developed by each participant (or pair). In about 5 of the 120 cases, the scores assigned by the raters differed significantly. These were reconciled through discussion. The final judgments of software quality by the raters were highly correlated (Pearson correlation = 0.983). To reduce any possible source of bias, the average of the scores by the two raters was used as a measure of *software quality*.

*Satisfaction* represents affective response of the individual to the overall task. This measure was adapted from Bhattacharjee (2001). The participants responded to the question "How do you feel about your overall experience of working on the programming task today?" using Likert scales ranging from (1) *very dissatisfied* to (7) *very satisfied*, (1) *very displeased* to (7) *very pleased*, (1) *very frustrated* to (7) *very contented*, and (1) *absolutely terrible* to (7) *absolutely delighted*. Items were summed and the mean score was used as the measure of *satisfaction* (Cronbach's  $\alpha = 0.934$ ).

*Confidence in performance* represents the strength of the participant's belief about the quality of his/her programming solution. The measure for this variable (Cronbach's  $\alpha = 0.945$ ) was adapted from existing literature (Brewer and Kramer 1986; Jourden and Heath 1996). It was designed to capture the subject's broad perception of his/her performance (quality of solution), to afford comparison with the more objective performance measure of software quality. Participants responded to the statement "How do you feel about the quality of your programming solution?" using Likert scales anchored by (1) *not at all confident* to (7) *very confident* and (1) *not at all certain* to (7) *very certain*. Participants also responded to the question "Imagine that we selected ten results at random from those who participated in this task. How would your performance rank among these ten results?" and ranked their performance on a Likert scale ranging from (1) *worst result out of ten* to (10) *best result out of ten*. As two items measured *confidence in performance* on a 7-point Likert scale and one item on a 10-point Likert scale, a summated scale was created for *confidence in performance* by adding the standardized individual items and then again standardizing the resultant summated variable. The resultant *confidence in performance* measure had a mean of 0 and standard deviation of 1.

## Control Variable

*Programming ability* is an important variable that can influence a participant's performance. To minimize the influence of this variable on performance, *programming ability* was measured and used as a covariate in the analysis. The index of *programming ability* was determined for each subject by computing his/her weighted average GPA in all Information Systems courses taken at the university. In this computation, grades in programming and analysis and design courses (the analysis and design course offered at the university involved implementation of a project in Java) were given twice the weight as those in other IS courses.

**Table 4. Factor Loadings of Perceptual Dependent Measures**

| Item                   | Satisfaction | Confidence in Performance | Communality Estimate $h_i^2$ |
|------------------------|--------------|---------------------------|------------------------------|
| 1                      | 0.843        | 0.362                     | 0.843                        |
| 2                      | 0.897        | 0.280                     | 0.883                        |
| 3                      | 0.843        | 0.300                     | 0.801                        |
| 4                      | 0.839        | 0.351                     | 0.827                        |
| 5                      | 0.323        | 0.886                     | 0.889                        |
| 6                      | 0.341        | 0.919                     | 0.961                        |
| 7                      | 0.349        | 0.914                     | 0.956                        |
| Eigenvalue $\lambda_j$ | 3.273        | 2.887                     | Total 6.16                   |
| Variance Explained     | 46.75%       | 41.24%                    | 87.99%                       |

### Preliminary Analysis

To check whether the scales measuring participant satisfaction and confidence were independent, exploratory factor analysis was conducted using orthogonal (Varimax) rotation. The items loaded highly on the right constructs, and had low loadings on other constructs, thus suggesting high convergent and discriminant validity. *Satisfaction* and *confidence in performance* had Eigenvalues of 3.27 and 2.89 respectively, and the two-factor solution represented 88 percent of the total variance. These results, therefore, confirmed *satisfaction* and *confidence in performance* to be distinct factors. Table 4 summarizes the results of factor analysis.

### Success of Experimental Manipulations

To verify the success of the experimental manipulation of collaborative working (i.e., the treatment group), the experimenter's log of visits to remind the collaborative pairs to take turns was checked. This confirmed that the experimenter delivered the instructions and that the subject did comply.

A two-item scale was used to verify the success of the manipulation of task complexity. Participants responded to the question "How do you feel about the main programming task you performed, as compared to the warm-up task?" using Likert scales from (1) *very easy* to (7) *very difficult* and (1) *very simple* to (7) *very complex*. Both items were summed and the mean scale score was computed ( $\alpha = 0.87$ ). Among nominal pairs, participants in each pair were grouped by their *software quality* scores into the best and the second-best performing members. The ANOVA procedure was used in a 3 *programming setting* (collaborating pair  $\times$  best nominal pair member  $\times$  second-best nominal pair member)  $\times$  2 *task com-*

*plexity* (high task complexity  $\times$  low task complexity) factorial design to verify the success of *task complexity* manipulation. Results indicated a main effect of *task complexity* manipulation for the scale measuring perceived complexity ( $F = 11.17$ ,  $p < .01$ ). Participants in the high task complexity conditions perceived the task to be more complex ( $M = 5.09$ ), compared with participants in the low task complexity conditions ( $M = 4.28$ ). No other main or interaction effects were significant, thus confirming that the experimental manipulation of *task complexity* was successful.

In addition, we also obtained independent evaluations of the tasks by two experts. Both experts hold academic positions and have several years of experience in teaching Java programming. One of the experts also has significant industry experience, having served as director of technology processes at a large multinational corporation prior to joining academia. Neither of the experts had anything to do with the experimental tasks. On the same set of questions indicated above, these experts also perceived the two experimental tasks to be different in complexity ( $M = 5.75$  for high complexity task and  $M = 3.75$  for low complexity task) relative to the warm-up task, providing further evidence of the success of the experimental manipulation.

### Analysis and Results

Among the dependent measures, *software quality* was measured as a group level construct for collaborating pairs, as each pair developed a single programming solution. The perceptual measures of participant *satisfaction* and *confidence in performance* were measured individually for each member of the collaborating pair and then averaged. Among nominal

pairs, participants in each pair were classified by their software quality scores into the best and the second-best performing members. To test the significance of experimental manipulations, MANCOVA procedures were used in a 3 *programming setting* (collaborating pair  $\times$  best nominal pair member  $\times$  second-best nominal pair member)  $\times$  2 *task complexity* (high task complexity  $\times$  low task complexity) factorial design, with *programming ability* used as a covariate. Testing for MANCOVA significance across dependent measures, before doing individual ANCOVA analyses, guards against inflated type I error (Hair et al. 1998). When MANCOVA tests are significant, ANCOVA procedures are recommended to determine which dependent variables are significant. Based on ANCOVA effects, *a priori* t-tests with Bonferroni's correction were used to examine the pattern of mean differences and to test specific hypotheses. Bonferroni's correction helps control for inflated type I error due to multiple comparisons.

As the experimental sessions were performed over three semesters, we conducted separate one-way ANOVA analyses to check for any systematic biases. The tests revealed no significant differences across semesters among the dependent measures of *software quality* ( $F(2, 87) = 0.54, p = 0.58$ ), *satisfaction* ( $F(2, 87) = 0.46, p = 0.63$ ), and *confidence in performance* ( $F(2, 87) = 1.01, p = 0.37$ ), providing assurance against any time-ordered effects. In terms of MANOVA assumptions, presence of significant correlations among dependent measures was satisfied based on Bartlett's test for sphericity ( $\chi^2 = 798.481, p < 0.01$ ). The linearity assumption was satisfied as no nonlinear relationships were evident in the scatter plots matrix of the dependent measures and the covariate. Based on Martinez and Iglewicz tests, the normality assumptions were satisfied for the dependent measures of *software quality* ( $I = 0.93, p > 0.05$ ), *satisfaction* ( $I = 0.98, p > 0.05$ ) and *confidence in performance* ( $I = 0.95, p > 0.05$ ). The assumption of equal variance across treatment groups was also satisfied, based on modified Levine test for *software quality* ( $F = 2.185, p = 0.063$ ), *satisfaction* ( $F = 0.863, p = 0.509$ ), and *confidence in performance* ( $F = 0.795, p = 0.557$ ), along with the assumption of equality of covariance matrices (Box's  $M = 31.730, p = 0.528$ ). It is pertinent to note that F test in ANOVA models is robust against violations of normality (Neter et al. 1996). Also, when sample sizes are roughly equal in all of the treatment cells, as is the case here, the effect of any violation of unequal variance assumption is minimal on the inference tests based on F-distribution (Hair et al. 1998; Neter et al. 1996).

MANCOVA procedures, used to examine the impact of the independent variables across the set of dependent variables (*software quality*, *satisfaction*, and *confidence in performance*), indicated that *programming setting* (collaborating

pairs  $\times$  best nominal pair members  $\times$  second-best nominal pair members) had a significant effect on the set of dependent measures (Wilke's Lambda = 0.64,  $F = 6.90, p < 0.01$ ), as did *task complexity* (Wilke's lambda = 0.91,  $F = 2.79, p = 0.046$ ), and *programming ability* (Wilke's Lambda = 0.80,  $F = 6.70, p < 0.01$ ). There was no significant interaction of *programming setting*  $\times$  *task complexity* on the set of dependent measures (Wilke's Lambda = 0.93,  $F = 1.04, p = 0.40$ ). Given the overall significance of the MANCOVA model, ANCOVA analyses were then conducted for each of the dependent measures. *Programming ability* was again used as a covariate in all of the analyses. The individual ANCOVA results for the three dependent measures are summarized in Table 5. Table 6 shows the means and standard deviations of dependent measures for the three conditions of *programming setting*. Figures 2 and 3 provide a graphical representation of the marginal means for these conditions.

## Software Quality

Hypothesis 1 predicted *software quality* scores to be higher for collaborating pairs, compared with the second-best members of nominal pairs. Based on ANCOVA procedures, there was a significant main effect of the programming setting on *software quality* ( $F(2, 83) = 12.73, p < 0.01$ ). T-tests were performed with Bonferroni's correction to check whether the pattern of mean differences was consistent with H1. Table 7 summarizes marginal means and results of the planned comparisons for Hypotheses 1 to 3. Hypothesis 1 was supported as *software quality* for collaborating pairs ( $M = 62.52$ ) was significantly higher than that of the second-best members of nominal pairs ( $M = 43.38$ ) in a one-tailed test of marginal means ( $p < 0.01$ ). Although not explicitly hypothesized, we did additional tests to more precisely locate the performance of the collaborating pair, relative to the performance of the best member of the nominal pair. Consistent with our expectation, there were no differences between the *software quality* scores of collaborating pairs ( $M = 62.52$ ) and those of the best members of nominal pairs ( $M = 73.03$ ) in a two-tailed test of marginal means ( $p = 0.16$ ).

Hypothesis 4 predicted that *task complexity*  $\times$  *programming setting* interaction influences *software quality*. Specifically, we expected *task complexity* to more adversely affect the *software quality* of the best members of the nominal pairs than that of the collaborating pairs. However, there was no support for Hypothesis 4, as ANCOVA procedures (Table 5) suggested that the interaction effect of *programming setting*  $\times$  *task complexity* on *software quality* was not significant ( $F(2, 83) = 1.73, p = 0.18$ ). The pattern of relationship between *programming setting* and *software quality* was, therefore, not affected by *task complexity*. There was a signi-

**Table 5. One-Way ANCOVA Results for The Dependent Measures**

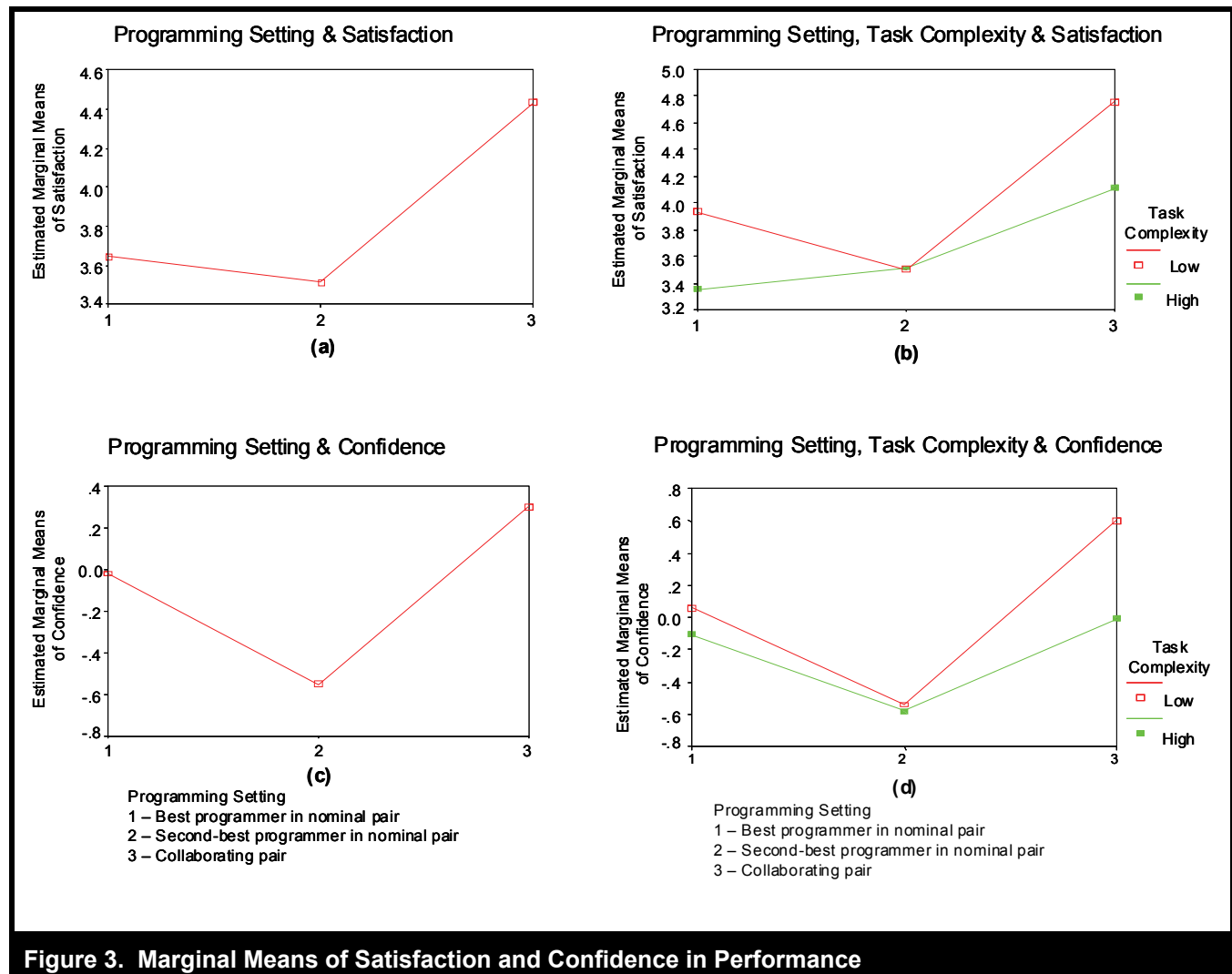
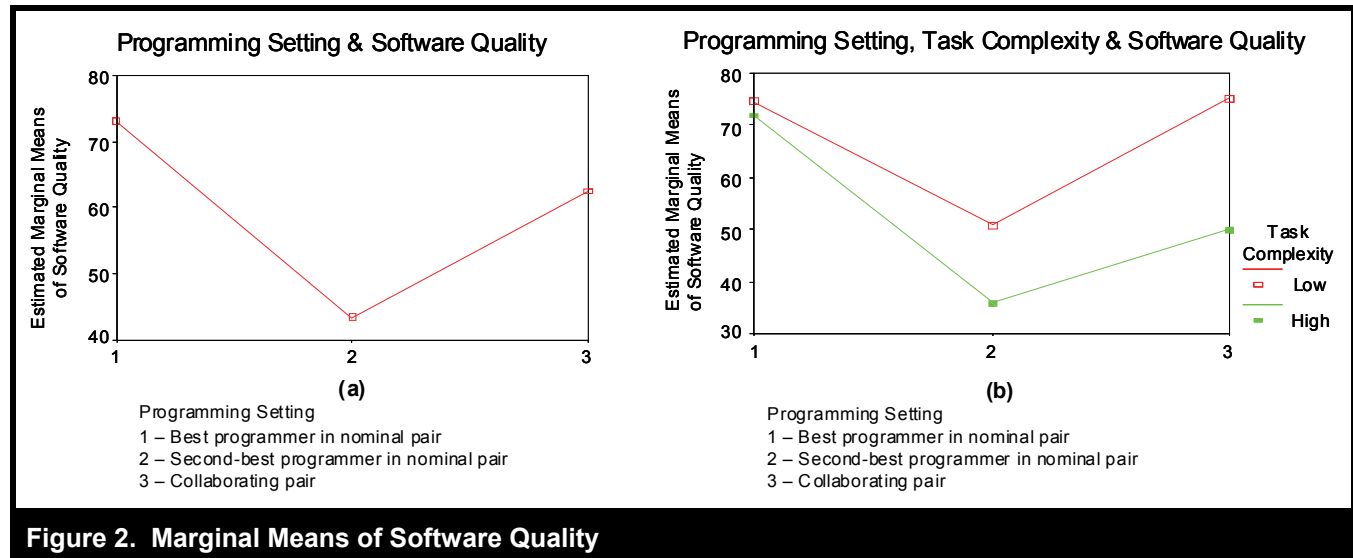
|                                                      | SS         | df | MS        | F      | p-value |
|------------------------------------------------------|------------|----|-----------|--------|---------|
| <b>Software Quality (SQ)</b>                         |            |    |           |        |         |
| Programming Setting (PS)                             | 13437.294  | 2  | 6718.647  | 12.731 | 0.000*  |
| Task Complexity (TC)                                 | 4510.399   | 1  | 4510.399  | 8.547  | 0.004*  |
| Programming Ability (GPA)                            | 10813.705  | 1  | 10813.705 | 20.490 | 0.000*  |
| PS × TC                                              | 1826.755   | 2  | 913.378   | 1.731  | 0.183   |
| Error                                                | 43802.995  | 83 | 527.747   |        |         |
| Total                                                | 396338.000 | 90 |           |        |         |
| Model R Squared = 0.425 (Adjusted R Squared = 0.383) |            |    |           |        |         |
| <b>Satisfaction (S)</b>                              |            |    |           |        |         |
| Programming Setting (PS)                             | 14.890     | 2  | 7.445     | 3.407  | 0.038*  |
| Task Complexity (TC)                                 | 3.611      | 1  | 3.611     | 1.653  | 0.202   |
| Programming Ability (GPA)                            | 8.503      | 1  | 8.503     | 3.891  | 0.052   |
| PS × TC                                              | 1.879      | 2  | 0.940     | 0.430  | 0.652   |
| Error                                                | 181.364    | 83 | 2.185     |        |         |
| Total                                                | 1552.547   | 90 |           |        |         |
| Model R Squared = 0.136 (Adjusted R Squared = 0.073) |            |    |           |        |         |
| <b>Confidence in Performance (CP)</b>                |            |    |           |        |         |
| Programming Setting (PS)                             | 11.200     | 2  | 5.600     | 7.781  | 0.001*  |
| Task Complexity (TC)                                 | 1.636      | 1  | 1.636     | 2.273  | 0.135   |
| Programming Ability (GPA)                            | 3.893      | 1  | 3.893     | 5.416  | 0.022*  |
| PS × TC                                              | 1.313      | 2  | 0.657     | 0.912  | 0.406   |
| Error                                                | 59.736     | 83 | 0.720     |        |         |
| Total                                                | 78.099     | 89 |           |        |         |
| Model R Squared = 0.235 (Adjusted R Squared = 0.180) |            |    |           |        |         |

\*significant at p = 0.05

**Table 6. Means and Standard Deviations for the Dependent Measures**

| Measures                                        | Nominal Pairs            |                 |               |                    |                 |               | Collaborating Pairs |                 |               |
|-------------------------------------------------|--------------------------|-----------------|---------------|--------------------|-----------------|---------------|---------------------|-----------------|---------------|
|                                                 | Best Person <sup>†</sup> |                 |               | Second-Best Person |                 |               |                     |                 |               |
|                                                 | Low Complexity           | High Complexity | Mean          | Low Complexity     | High Complexity | Mean          | Low Complexity      | High Complexity | Mean          |
| Software Quality (Range 0–125)                  |                          |                 |               |                    |                 |               |                     |                 |               |
| Mean                                            | 77.400                   | 72.600          | <b>75.000</b> | 47.633             | 37.267          | <b>42.450</b> | 73.767              | 49.200          | <b>61.483</b> |
| SD                                              | 21.706                   | 24.603          | <b>22.927</b> | 24.161             | 22.137          | <b>23.371</b> | 28.519              | 30.619          | <b>31.644</b> |
| n                                               | 15                       | 15              | <b>30</b>     | 15                 | 15              | <b>30</b>     | 15                  | 15              | <b>30</b>     |
| Satisfaction (Range 1–7)                        |                          |                 |               |                    |                 |               |                     |                 |               |
| Mean                                            | 4.017                    | 3.383           | <b>3.700</b>  | 3.417              | 3.550           | <b>3.483</b>  | 4.717               | 4.092           | <b>4.404</b>  |
| SD                                              | 1.483                    | 1.700           | <b>1.600</b>  | 1.764              | 1.240           | <b>1.500</b>  | 1.181               | 1.557           | <b>1.395</b>  |
| n                                               | 15                       | 15              | <b>30</b>     | 15                 | 15              | <b>30</b>     | 15                  | 15              | <b>30</b>     |
| Confidence in Performance (Standardized Scores) |                          |                 |               |                    |                 |               |                     |                 |               |
| Mean                                            | 0.118                    | -0.087          | <b>0.157</b>  | -0.597             | -0.551          | <b>-0.574</b> | 0.578               | -0.020          | <b>0.279</b>  |
| SD                                              | 0.846                    | 0.985           | <b>0.908</b>  | 0.989              | 0.675           | <b>0.832</b>  | 0.769               | 0.912           | <b>0.883</b>  |
| n                                               | 15                       | 15              | <b>30</b>     | 15                 | 15              | <b>30</b>     | 15                  | 15              | <b>30</b>     |

<sup>†</sup>The best (most competent) person and the second best (least competent) person of each nominal pair were determined based on their software quality scores.



**Table 7. Marginal Means and Planned Comparison Tests for Hypotheses 1, 2 and 3**

| Measure          |    | Nominal Pair |                    | Collaborating Pair | Hypotheses         | p value |
|------------------|----|--------------|--------------------|--------------------|--------------------|---------|
|                  |    | Best Member  | Second-best Member |                    |                    |         |
|                  |    | 1            | 2                  | 3                  |                    |         |
| Software Quality | SQ | 73.033       | 43.383             | 62.517             | H1: SQ3 – SQ2 > 0  | 0.002*  |
| Satisfaction     | S  | 3.645        | 3.509              | 4.433              | H2A: S3 – S1 > 0   | 0.043*  |
|                  |    |              |                    |                    | H2B: S3 – S2 > 0   | 0.018*  |
| Confidence in    | CP | -0.022       | -0.556             | 0.299              | H3A: CP3 – CP1 > 0 | 0.150   |
| Performance      |    |              |                    |                    | H3B: CP3 – CP2 > 0 | 0.000*  |

\*significant at  $p = 0.05$

ficant main effect of *task complexity* on *software quality* ( $F(1, 83) = 8.55, p < 0.01$ ). A t-test with Bonferroni's correction on marginal means suggested the mean *software quality* score was lower ( $p < 0.01$ ) for the high complexity task ( $M = 52.56$ ) compared with the low complexity task ( $M = 66.73$ ).

### Satisfaction

Based on ANCOVA analysis (Table 5), there was also a significant main effect of *programming setting* on *satisfaction* ( $F(2, 83) = 3.41, p = 0.04$ ). Hypotheses 2A and 2B predicted *satisfaction* levels of collaborating pairs to be higher than those of the best and the second-best members of nominal pairs. Both hypotheses were supported. The *satisfaction* reported by the collaborating pairs ( $M = 4.43$ ) was significantly higher than those expressed by the best ( $M = 3.65, p = 0.04$ ) and the second-best ( $M = 3.51, p = 0.02$ ) members of nominal pairs in one-tailed t-tests of marginal means. Although we did not have any prior expectation, the interaction of *programming setting*  $\times$  *task complexity* on *satisfaction* was not significant ( $F(2, 83) = 0.43, p = 0.65$ ).

### Confidence in Performance

ANCOVA results indicated a significant main effect of *programming setting* manipulation on *confidence in performance* ( $F(2, 83) = 7.78, p < 0.01$ ). Hypotheses 3A and 3B predicted *confidence in performance* levels to be higher for collaborating pairs compared with those of the best and the second-best members of nominal pairs. There was no support for Hypothesis 3A, as the confidence levels reported were not significantly higher for the collaborating pairs relative to the best members of nominal pairs in a one-tailed t-test of marginal means ( $p = 0.15$ ). However, there was support for

Hypothesis 3B, as collaborating pairs had significantly higher confidence levels compared with the second-best members of nominal pairs in a one-tailed t-test ( $p < 0.01$ ). Although we did not have any prior expectation, the interaction of *programming setting*  $\times$  *task complexity* on confidence levels of participants was not significant ( $F(2, 83) = 0.91, p = 0.41$ ).

To summarize, four main hypotheses in all were fully supported. Two hypotheses (Hypothesis 3A and 4) were not supported. The results of hypothesis testing are summarized in Table 8. A complete discussion of the results and the plausible reasons for exceptions are presented in the next section.

## Discussion

In this paper, we examined the effects of paired programmers on performance outcomes by contrasting them with individual programmers, whose efforts were combined into nominal pairs. The nominal pairs in this study were pairs that were randomly created. Members of each nominal pair worked alone, but their efforts were conjointly considered. To the best of our knowledge, no previous IS study has attempted to compare the performance of collaborating programming pairs with that of nominal pairs. When performance of collaborating pairs is contrasted with an equal number of individual programmers, as in previous IS studies, the comparison inherent in such designs is between pair performance and the performance of an average individual programmer. Pairs are expected to outperform average individuals based on pure probabilities alone. By using the same number of programmers in the individual condition as in the paired condition (e.g., 30 pairs and 60 individuals) and by statistically grouping the individual programmers into nominal pairs (e.g., 30 nominal pairs), this study created an equal man-hour comparison between programmers working collaboratively in

**Table 8. Results of Hypotheses Testing**

|   | Hypothesis                                                                                                                                                                   | Result                      | Explanation                                                                                                                                                               |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Software Quality</b>                                                                                                                                                      |                             |                                                                                                                                                                           |
|   | <i>Programming performance of a collaborating pair measured in terms of software quality will be higher than the performance of the second-best member of a nominal pair</i> | Supported<br>(p < 0.01)     | A collaborating pair outperforms the second-best member of an independently working nominal pair.                                                                         |
| 2 | <b>Satisfaction</b>                                                                                                                                                          |                             |                                                                                                                                                                           |
|   | <i>A. The mean satisfaction with the programming task of a collaborating pair will exceed the level reported by the best member of a nominal pair.</i>                       | Supported<br>(p = 0.04)     | A collaborating pair is more satisfied than each member of an independently working nominal pair.                                                                         |
|   | <i>B. The mean satisfaction with the programming task of a collaborating pair will exceed the level reported by the second-best member of a nominal pair.</i>                | Supported<br>(p = 0.02)     |                                                                                                                                                                           |
| 3 | <b>Confidence in Performance</b>                                                                                                                                             |                             |                                                                                                                                                                           |
|   | <i>A. The mean confidence in performance of a collaborating programming pair will exceed the level reported by the best member of a nominal pair.</i>                        | Not Supported<br>(p = 0.15) | A collaborating pair is only as confident of its performance as the best member of an independently working nominal pair, but more confident than its second-best member. |
|   | <i>B. The mean confidence in performance of a collaborating programming pair will exceed the level reported by the second-best member of a nominal pair.</i>                 | Supported<br>(p < 0.01)     |                                                                                                                                                                           |
| 4 | <b>Moderating Effect of Task Complexity</b>                                                                                                                                  |                             |                                                                                                                                                                           |
|   | <i>Task complexity affects the performance, in terms of software quality, of the best member of a nominal pair more adversely than that of a collaborating pair</i>          | Not Supported<br>(p = 0.18) | Task complexity affects the performance of collaborating pairs and nominal pairs in similar ways.                                                                         |

pairs and those working alone. This is consistent with recent designs used in the social psychology literature (Laughlin, Zander et al. 2003). Drawing not only from the IS field, but also from the literature on social and cognitive psychology, hypotheses comparing collaborating pairs with nominal pairs, in terms of performance outcome and affective responses, were derived. The findings are generally supportive of the research model and the main hypotheses.

### Software Quality

The results indicated that while working on a programming task, performance, in terms of *software quality*, of a collaborating pair is higher than that of the second-best programmer of a nominal pair working independently, but is not different from the performance of the best member of the nominal pair. Therefore, no *assembly bonus effect*, where pair performance exceeds that of the most competent member of a nominal pair,

was found among the participants in this study. Although consistent with our expectation, it runs counter to some of the earlier findings in XP literature where there is a popular and general perception of pair superiority over individuals. Such perceptions may stem from the known superiority of groups over average individuals in problem-solving tasks, and is perhaps enhanced by the positive affect experienced by group members during collaborative work (Hinsz and Nickell 2004).

Our results, on pair programming performance, are in line with the general finding in small group research that groups rarely outperform the best individual of a statistical aggregate (Hill 1982; Kerr and Tindale 2004). Additionally, Laughlin and colleagues suggested that in intellectual problem solving tasks, a group size of three is necessary for the group to outperform the best individual of a nominal group (Laughlin, Hatch et al. 2006). The consistency of our finding with the research evidence from small group studies provides assurance regarding the plausibility and robustness of the underlying effect.

Despite the popular perception among the software community of pair superiority, performance above the most competent member of a collaborating pair may be difficult to achieve in pair programming because of the nature of the programming task itself. As brought out earlier, programming tasks do have intellectual components with correct solutions, but the demonstrability of such solutions decreases with increasing task complexity. In addition, there are decision-making aspects to programming tasks that involve inherent tradeoffs and require evolving a consensus on a preferred approach. This also adds to the coordination overhead, thus affecting pair performance. Evidence from group brainstorming research suggests that collaborating groups fall short of producing as many ideas as an equivalent number of individuals in nominal groups (Mullen and Salas 1991). In what is termed as production blocking, wait periods involved in turn taking could cause cognitive interference (Diehl and Stroebe 1987). Similar production blocking processes may be at play when programmers switch roles in working with XP procedures, thus reducing potential gains from collaboration. One possible way for the pairs to reduce this interference is to develop effective work routines over long periods of working together.

### **Task Complexity**

Performance differences between collaborating pairs and the best as well as the second best performers in nominal pairs were found to be consistent across tasks of different levels of complexity. That is, task complexity was not found to have a moderating effect on these relationships when software quality was considered. Instead, task complexity had a direct effect on participants: those (both individuals and pairs) working on tasks of high complexity scored lower on software quality relative to participants working on tasks of low complexity.

With increasing task complexity, performance of a collaborating pair could theoretically improve up to the point where it matches the cognitive capacity of the pair. Beyond this point, performance would be expected to deteriorate. It is possible that in the present study, the more complex task used was beyond this optimal cognitive capacity for some groups, and hence the hypothesized moderating effect of task complexity was not evident. The observed direct effect of performance deterioration of participants working on the more complex task also supports our speculation in this regard. However, this issue needs to be further investigated in future studies.

### **Satisfaction and Confidence**

The results of this study suggest that collaborating pairs were more satisfied than the best and the second-best members of nominal pairs. These findings are consistent with empirical evidence from IS and small group research that group work is more satisfying. It is interesting to note that members of collaborating pairs showed higher levels of satisfaction, irrespective of their performance as measured in terms of software quality.

Collaborating pairs also had more confidence in their performance than did the second-best members. However, collaborating pairs reported only as much confidence in their performance as the best members of nominal pairs. The confidence levels were in correspondence with the performance levels, as collaborating pairs outperformed the second-best members of nominal pairs in software quality achieved, but not the best members. This is suggestive of realism in the confidence judgments of programming pairs. Findings in small group research suggest that groups tend to have more positive illusions of their performance than do individuals (Jourden and Heath 1996). Our findings indicate that, at least in the programming context, pairs are able to avoid this over-performance bias. The ability to make realistic confidence judgments augurs well for programming pairs, because pairs would then accurately judge their performance and appropriately respond to changing situational requirements.

### **Limitations**

The types of software developers simulated in this study were entry-level programmers engaged in systems development in organizations. One concern when using student subjects would be with regard to their representativeness of the target population. The average self-reported programming experience of the subjects involved in this study was 1.92 years, with 38 percent reporting 2 or more years of programming experience. This alleviates some concerns that our participants were unfamiliar with programming demands. Another issue is that student programmers, as participants, may have difficulty remembering the syntax of the programming language. However, we did provide access to online documentation, although it is hard to say if they were more or less motivated to use such documentation than participants in other situations. Also, it is not uncommon to find student subjects being used as surrogates for practitioners in other IS studies (see, for example, Bodart et al. 2001; Khatri et al. 2006; Kumar and Benbasat 2004). Other issues concern the many complex situational variables of the work place that are not present in the more sterile laboratory environment.

However, we would suggest that the underlying cognitive and behavioral processes reflected in these findings operate very similarly in the workplace. Also, social psychologists, who have examined the effectiveness of individuals and groups on problem-solving tasks, have reported similar results (e.g., Laughlin, Gonzalez et al. 2003; Laughlin, VanderStoep and Hollingshead 1991).

Another potential limitation of the paper is the use of GPA as an index of programming ability. However, it must be mentioned that, when the GPA was calculated, courses that involved programming in Java were given twice as much weight as other classes. This weighted GPA, rather than a straightforward one, was a covariate that significantly accounted for the variance of both *software quality* ( $p < 0.01$ ) and *confidence in performance* ( $p < 0.022$ ), thus lending credence to our approach.

Some researchers have suggested that pair performance improves after an initial jelling period (Williams et al. 2000). This was not simulated in the present study, except for working on a warm-up task of 15 minutes duration. This study also did not measure task completion time. It would be a useful dependent measure in a study where performance is defined as successful completion to a particular standard. As software quality was conceived as a continuous measure, time to completion was used as a control factor in our study, with all participants provided with a maximum time of 2 hours for task completion.

### **Implications for Research and Practice**

This study has made valuable contributions to research in the domains of software development, IS group problem-solving, and social psychology. For software development research in general, and agile development in particular, this study has suggested that when working on software maintenance tasks, irrespective of task complexity, the performance of a programmer pair exceeds that of the second-best member of a nominal group, but does not exceed that of the best member.

Furthermore, this study has introduced the concept of the nominal group and the performance of its best member as a new benchmark for evaluating group problem solving tasks in software development. Nominal group design, which is widely used in social psychology studies, offers a rigorous approach to compare group performance with the performances of individuals. For instance, in four-member groups, comparison is possible between performance of the group and the performances of best, second-best, third-best, and fourth-best members of a nominal group.

In the social psychology literature, understanding the relationship between the task type, skills and abilities of the group members, and the performance of the group has been of enduring interest. Typically, the tasks used in these studies fall into either end of an intellectual-judgmental continuum. When the task is highly intellectual, as in studies by Laughlin and his colleagues (e.g., Laughlin, Bonner and Miner 2002; Laughlin, Hatch et al. 2006; Laughlin, Zander et al. 2003), a truth-wins social combination process (where a group's response is the correct response if any one member can solve the problem) often defines the group's performance. With increasing task complexity, the correct solution becomes less obvious where other social combination processes, such as truth-supported (where a pair can come up with a correct response only if both members can agree on the correct solution), define group performance. Programming tasks, while combining multiple disjunctive subtasks which might facilitate the contributions of multiple members, however, also combine intellectual with decision-making task components, where consensus, rather than identification of the correct response, defines the solution. By examining group problem solving on a programming task, this paper has extended this stream of research into a type of task that is complex, with multiple and conjoint elements. In addition, this type of task also comprises a core activity performed by employees in many organizations, and thus has high practical relevance. The idea of nominal groups is relatively new in studies involving group problem solving and, unlike the tradition of other established concepts in group theory, the cumulative body of research on nominal groups is relatively small. Our study would no doubt be a useful addition to this growing body of work.

For enhancing the practice of pair programming in organizations, this study has made several contributions. First, pair programming helps a pair to produce software of superior quality compared with that of its less competent member, and that pair performance in this dimension is comparable to its best member working individually. Thus, there is no downside to pair programming, in terms of the quality of software produced, although the cost of an additional programmer needs to be critically examined. It is said that for effective software development, developers' experience of building systems is important in both agile and traditional software development groups. However, individual competency of developers is a critical success factor for agile development methodologies (Cockburn and Highsmith 2001). We would, therefore, speculate that an organization using pair programming as a practice could benefit more by pairing programmers from its available pool of those who rank above and below the median, in terms of abilities. This could be viewed as an insurance mechanism to achieve software quality comparable to that produced by the top half performers.

Second, pair programming contributes to increased *satisfaction* over independent working. Higher satisfaction of programming pairs has implications for reducing employee turnover in general, and improving retention of best performers in particular.

Third, based on the findings of this study, there is an imperative need to critically evaluate pair programming as a common XP practice for all aspects of software development. Consistent with the general finding of several decades of small group research, this study has shown that pair performance cannot exceed the performance of the best programmer in the dyad working alone. So, while pair programming is definitely an option when working on mission critical aspects of software development, the use of this practice needs to be carefully evaluated so that expectations do not exceed the typical benefits of the strategy.

### Future Research Directions

This study has introduced a new performance benchmark, performance above the best member working alone, for judging agile software development procedures. It would be useful in future studies to see how this performance benchmark could be breached, if at all. The quest for the *assembly bonus effect* in pair programming would be an interesting and rewarding research program to pursue. This entails an examination of individual differences between members of pairs, as well as an investigation of agile software development issues related to tasks, context, and process.

The programming tasks used in our study were software maintenance tasks, where subjects made enhancements to an existing code. Among various software development tasks, such as design, implementation, testing, and maintenance, maintenance tasks may have the lowest creative potential. That is, they may provide a class of problems that are more likely to have a verifiable component relative to design or implementation tasks. Consequently, it would be interesting to see whether these findings would replicate to other types of software development tasks. While we found the main effect to be consistent across tasks of two levels of complexity, it would be of interest to compare across, say, more than two levels of complexity, as we speculate a nonlinear moderating relationship for task complexity.

Solution demonstrability increases the likelihood that pairs will adopt the correct answer suggested by one of its members. This process involves not only task characteristics, but also person characteristics, as the dyad member suggesting a solution should have the ability, motivation, and time to

demonstrate the correct answer to the partner. A recent case study finding suggests that novice–novice pairs, relative to novice solos, are more productive than expert–expert pairs, relative to expert solos (Lui and Chan 2006). Examining such performance effects of pairing programmers with different combinations of abilities, or even personalities, would be an exciting avenue to pursue with interesting insights to be gained.

## Conclusions

This study makes significant contributions to understanding the performance effectiveness of pair programming by demonstrating, through a controlled experiment, that pair performance typically cannot exceed the performance of its best member working individually. No effect was evident for task complexity on this relationship. In terms of the affective outcomes, pairs were more satisfied than both the best and the second-best members of nominal pairs. Also, pairs were more confident in their performance, compared to the second-best members of nominal pairs, but not the best members. These results were obtained by contrasting collaborating pairs with nominal pairs, based on an equal man-hour comparison.

With the advent of agile methodologies, software development is undergoing an extreme makeover. Software practice is in need of robust research findings to wade through the methodological quagmire facing the software development community. An enlightened perspective is, however, needed that stresses rigorous and generalizable research to avoid deleterious consequences, such as wasted developer productivity, software project failures, and operational disruptions caused by defective software. In attempting to achieve this ideal, this paper makes a valuable contribution to the current discourse on collaborative practices in agile development methodologies in general, and the practice of pair programming in particular.

### Acknowledgments

The authors thank Radhika Santhanam and James Teng for their valuable comments. The authors are also grateful to the anonymous reviewers, the associate editor, and Carol Saunders for their constructive feedback and suggestions on earlier versions of this paper.

### References

- Aiello, J. R., and Douthitt, E. A. 2001. "Social Facilitation from Triplett to Electronic Performance Monitoring," *Group Dynamics* (5:3), pp. 163-180.

- Ambler, S. 2007. "Survey Says...Agile Has Crossed the Chasm," Dr. Dobb's Portal: Architecture & Design, July 2 (<http://www.ddj.com/architect/200001986>).
- Arisholm, E., Gallis, H., Dybå, T., and Sjøberg, D. I. K. 2007. "Evaluating Pair Programming with Respect to System Complexity and Programmer Expertise," *IEEE Transactions on Software Engineering* (33:2), pp. 65-86.
- Baheti, P., Gehringer, E., and Stotts, D. 2002. "Exploring the Efficacy of Distributed Pair Programming," in *Extreme Programming and Agile Methods - XP/Agile Universe 2002*, D. Wells and L. Williams (eds.), Heidelberg: Springer-Verlag, pp. 208-220.
- Beck, K. 1999. "Embracing Change with Extreme Programming," *IEEE Computer* (32:10), pp. 70-77.
- Beck, K. 2000. *Extreme Programming Explained: Embrace Change*, Reading, MA: Addison Wesley.
- Bhattacharjee, A. 2001. "Understanding Information Systems Continuance: An Expectation-Confirmation Model," *MIS Quarterly* (25:3), pp. 351-370.
- Bodart, F., Patel, A., Sim, M., and Weber, R. 2001. "Should Optional Properties Be Used in Conceptual Modeling? A Theory and Three Empirical Tests," *Information Systems Research* (12:4), pp. 384-405.
- Boland, R. J., and Tenkasi, R. V. 1995. "Perspective Making and Perspective Taking in Communities of Knowing," *Organization Science* (6:4), pp. 350-372.
- Bonner, B. L. 2004. "Expertise in Group Problem Solving: Recognition, Social Combination, and Performance," *Group Dynamics: Theory, Research, and Practice* (8:4), pp. 277-290.
- Bonner, B. L., Baumann, M. R., and Dalal, R. S. 2002. "The Effects of Member Expertise on Group Decision-Making and Performance," *Organizational Behavior and Human Decision Processes* (88:2), pp. 719-736.
- Brewer, M. B., and Kramer, R. M. 1986. "Choice Behavior in Social Dilemmas: Effects of Social Identity, Group Size, and Decision Framing," *Journal of Personality and Social Psychology* (50:3), pp. 543-549.
- Brickner, M. A., Harkins, S. G., and Ostrom, T. M. 1986. "Effects of Personal Involvement: Thought-Provoking Implications for Social Loafing," *Journal of Personality and Social Psychology* (51), pp. 763-770.
- Bystrom, K., and Jarvelin, K. 1995. "Task Complexity Affects Information-Seeking and Use," *Information Processing & Management* (31:2), pp. 191-213.
- Campbell, D. J. 1988. "Task Complexity: A Review and Analysis," *Academy of Management Review* (13:1), pp. 40-52.
- Canfora, G., Cimitile, A., Garcia, F., Piattini, M., and Visaggio, C. A. 2007. "Evaluating Performances of Pair Designing in Industry," *Journal of Systems and Software* (80:8), pp. 1317-1327.
- Canfora, G., Cimitile, A., and Visaggio, C. A. 2005. "Empirical Study on the Productivity of the Pair Programming," in *Extreme Programming and Agile Processes in Software Engineering*, H. Baumeister, M. Marchesi and M. Holcombe (eds.), Berlin: Springer, pp. 92-99.
- Cao, L., Mohan, K., Xu, P., and Ramesh, B. 2004. "How Extreme Does Extreme Programming Have to Be? Adapting XP Practices to Large-Scale Projects," in *Proceedings of the 37th Annual Hawaii International Conference on System Sciences*, Los Alamitos, CA: IEEE Computer Society Press.
- Cockburn, A. 2002. "Agile Software Development Joins the 'Would-Be' Crowd," *Cutter IT Journal* (15:1), pp. 6-12.
- Cockburn, A., and Highsmith, J. 2001. "Agile Software Development 2: The People Factor," *IEEE Computer* (34:11), pp. 131-133.
- Cockburn, A., and Williams, L. 2001. "The Costs and Benefits of Pair Programming," in *Extreme Programming Examined*, G. Succi and M. Marchesi (eds.), Boston: Addison Wesley, pp. 223-243.
- Collins, B. E., and Guetzkow, H. G. 1964. *Social Psychology of Group Processes for Decision-Making*, New York: Wiley.
- Dennis, A. R., and Valacich, J. S. 1993. "Computer Brainstorms: More Heads Are Better than One," *Journal of Applied Psychology* (78:4), pp. 531-537.
- Diehl, M., and Stroebe, W. 1987. "Productivity Loss in Idea-Generating Groups: Tracking Down the Blocking Effect," *Journal of Personality and Social Psychology* (53:3), pp. 497-509.
- Dybå, T., Arisholm, E., Sjøberg, D. I. K., Hannay, J. E., and Shull, F. "Are Two Heads Better than One? On the Effectiveness of Pair Programming," *IEEE Software* (24:6), pp. 12-15.
- Flor, N. V., and Hutchins, E. L. 1991. "Analyzing Distributed Cognition in Software Teams: A Case Study of Team Programming During Perfective Software Maintenance," in *Proceedings of the 4th Annual Workshop on Empirical Studies of Programmers*, J. Koenemann-Belleveau, T. G. Moher, and S. P. Robertson (eds.), Norwood, NJ: Ablex Publishing, pp. 36-63.
- Forsyth, D. R. 1999. *Group Dynamics*, Belmont, CA: Wadsworth Publishing.
- Fruhling, A., and de Vreede, G. 2006. "Field Experiences with eXtreme Programming: Developing an Emergency Response System," *Journal of Management Information Systems* (22:4), pp. 39-68.
- Garibaldi, A. M. 1979. "Affective Contributions of Cooperative and Group Goal Structures," *Journal of Educational Psychology* (71:6), pp. 788-794.
- Hackman, J. R. 1987. "The Design of Work Teams," in *Handbook of Organizational Behavior*, J. W. Lorsch (ed.), Englewood Cliffs, NJ: Prentice-Hall, pp. 315-342.
- Hair, J. F., Anderson, R. E., Tatham, R. L., and Black, W. C. 1998. *Multivariate Data Analysis*, Upper Saddle River, NJ: Prentice-Hall.
- Harkins, S. G. 1987. "Social Loafing and Social Facilitation," *Journal of Experimental Social Psychology* (23:11), pp. 1-18.
- Harkins, S. G., and Petty, R. E. 1982. "Effects of Task Difficulty and Task Uniqueness on Social Loafing," *Journal of Personality and Social Psychology* (43:6), pp. 1241-1229.
- Heiberg, S., Puus, U., Salumaa, P., and Seeba, A. 2003. "Pair-Programming Effect on Developers Productivity," in *Extreme Programming and Agile Processes in Software Engineering*,

- M. Marchesi and G. Succi (eds.), Berlin: Springer-Verlag, pp. 215-224.
- Highsmith, J., and Cockburn, A. 2001. "Agile Software Development 1: The Business of Innovation," *IEEE Computer* (34:9), pp. 120-127.
- Hill, G. W. 1982. "Group Versus Individual Performance: Are N + 1 Heads Better than One?," *Psychological Bulletin* (91:3), pp. 517-539.
- Hinsz, V. B. 1995. "Goal Setting by Groups Performing an Additive Task: A Comparison with Individual Goal Setting," *Journal of Applied Social Psychology* (25:11), pp. 965-990.
- Hinsz, V. B., and Nickell, G. S. 2004. "Positive Reactions to Working in Groups in a Study of Group and Individual Goal Decision Making," *Group Dynamics* (8:4), pp. 253-264.
- Hinsz, V. B., Tindale, R. S., and Vollrath, D. A. 1997. "The Emerging Conceptualization of Groups as Information Processors," *Psychological Bulletin* (121:1), pp. 43-64.
- Hulkko, H., and Abrahamsson, P. 2005. "A Multiple Case Study on the Impact of Pair Programming on Product Quality," *Proceedings of the 27<sup>th</sup> International Conference on Software Engineering*, St. Louis, MO, pp. 495-504.
- Ito, J. K., and Peterson, R. B. 1986. "Effects of Task Difficulty and Interunit Interdependence on Information Processing Systems," *Academy of Management Journal* (29:1), pp. 139-149.
- ITPro. 2007. "IT Spending Growth Expected to Slow," *ITPro News Briefs*, January/February, p. 7 (<http://csdl2.computer.org/comp/mags/it/2007/01/f1006.pdf>).
- Jourden, F. J., and Heath, C. 1996. "The Evaluation Gap in Performance Perceptions: Illusory Perceptions of Groups and Individuals," *Journal of Applied Psychology* (81:4), pp. 369-379.
- Karau, S. J., and Williams, K. D. 1993. "Social Loafing: A Meta-Analytic Review and Theoretical Integration," *Journal of Personality and Social Psychology* (65:4), pp. 681-706.
- Kerr, N. L., and Tindale, R. S. 2004. "Group Performance and Decision Making," *Annual Review of Psychology* (55:1), pp. 623-655.
- Khatri, V., Vessey, I., Ram, S., and Ramesh, V. 2006. "Cognitive Fit Between Conceptual Schemas and Internal Problem Representations: The Case of Geospatio-Temporal Conceptual Schema Comprehension," *IEEE Transactions on Professional Communication* (49:2), pp. 109-127.
- Kumar, N., and Benbasat, I. 2004. "The Effect of Relationship Encoding, Task Type, and Complexity on Information Representation: An Empirical Evaluation of 2D and 3D Line Graphs," *MIS Quarterly* (28:2), pp. 255-281.
- Latane, B., Williams, K. D., and Harkins, S. G. 1979. "Many Hands Make Light the Work: The Causes and Consequences of Social Loafing," *Journal of Personality and Social Psychology* (37:6), pp. 822-832.
- Laughlin, P. R. 1980. "Social Combination Processes of Cooperative Problem-Solving Groups on Verbal Intellectual Tasks," in *Progress in Social Psychology*, M. Fishbein (ed.), Hillsdale, NJ: Erlbaum, pp. 127-155.
- Laughlin, P. R., Bonner, B. L., and Altermatt, T. W. 1998. "Collective Versus Individual Induction with Single Versus Multiple Hypotheses," (75:6), pp. 1481-1489.
- Laughlin, P. R., Bonner, B. L., and Miner, A. G. 2002. "Groups Perform Better than the Best Individuals on Letter-to-Numbers Problems," *Organizational Behavior and Human Decision Processes* (88:2), pp. 605-620.
- Laughlin, P. R., and Ellis, A. L. 1986. "Demonstrability and Social Combination Processes on Mathematical Intellectual Tasks," *Journal of Experimental Social Psychology* (22:3), pp. 177-189.
- Laughlin, P. R., Gonzalez, C. M., and Sommer, D. 2003. "Quantity Estimations by Groups and Individuals: Effects of Known Domain Boundaries," *Group Dynamics* (7:1), pp. 55-63.
- Laughlin, P. R., Hatch, E. C., Silver, J. S., and Boh, L. 2006. "Groups Perform Better Than the Best Individuals on Letters-to-Numbers Problems: Effects of Group Size," *Journal of Personality and Social Psychology* (90:4), pp. 644-651.
- Laughlin, P. R., VanderStoep, S. W., and Hollingshead, A. B. 1991. "Collective Versus Individual Induction: Recognition of Truth, Rejection of Error, and Collective Information Processing," *Journal of Personality and Social Psychology* (61:1), pp. 50-67.
- Laughlin, P. R., Zander, M. L., Knievel, E. M., and Tan, T. K. 2003. "Groups Perform Better Than the Best Individuals on Letters-to-Numbers Problems: Informative Equations and Effective Strategies," *Journal of Personality and Social Psychology* (85:4), pp. 684-694.
- Levine, J. M., and Moreland, R. L. 1998. "Small Groups," in *Handbook of Social Psychology*, D. T. Gilbert and S. T. Fiske (eds.), New York: McGraw-Hill, pp. 415-469.
- Lui, K. M., and Chan, K. C. C. 2005. "Pair Programming Productivity: Novice-Novice vs. Expert-Expert," *International Journal of Human-Computer Studies* (64:9), pp. 915-925.
- Madeyski, L. 2006. "The Impact of Pair Programming and Test-Driven Development on Package Dependencies in Object-Oriented Design—An Experiment," in *Product-Focused Software Process Improvement*, J. Munch and M. Vierimaa (eds.), Heidelberg: Springer-Verlag, pp. 278-289.
- Madeyski, L. 2007. "On the Effects of Pair Programming on Thoroughness and Fault-Finding Effectiveness of Unit Tests," in *Product-Focused Software Process Improvement*, J. Munch and P. Abrahamsson (eds.), Heidelberg: Springer-Verlag, pp. 207-221.
- Maslow, A. H. 1943. "A Theory of Human Motivation," *Psychological Review* (50:4), pp. 370-396.
- McDowell, C., Werner, L., Bullock, H. E., and Fernald, J. 2006. "Pair Programming Improves Student Retention, Confidence, and Program Quality," *Communications of the ACM* (49:8), pp. 90-95.
- McGrath, J. E. 1984. *Groups: Interaction and Performance*, Englewood Cliffs, NJ: Prentice-Hall, Inc.
- Mullen, B., and Salas, J. C. 1991. "Productivity Loss in Brainstorming Groups: A Meta-Analytic Integration," *Basic Applied Social Psychology* (12:1), pp. 3-23.
- Müller, M. M. 2005. "Two Controlled Experiments Concerning the Comparison of Pair Programming to Peer Review," *Journal of Systems and Software* (78:2), pp. 166-179.
- Müller, M. M. 2006. "A Preliminary Study on the Impact of a Pair Design Phase on Pair Programming and Solo Programming," *Information and Software Technology* (48:5), pp. 335-344.

- Nawrocki, J., and Wojciechowski, A. 2001. "Experimental Evaluation of Pair Programming," in *Proceedings of the 12<sup>th</sup> European Software Control and Metrics Conference*, London, pp. 269-276.
- Neter, J., Kutner, M. H., Nachtsheim, C. J., and Wasserman, W. 1996. *Applied Linear Models*, Chicago: Irwin, 1996.
- Newell, A., and Simon, H. A. 1972. *Human Problem Solving*, Englewood Cliffs, NJ: Prentice-Hall.
- Nixon, H. L. 1979. *The Small Group*, Englewood Cliffs, NJ: Prentice-Hall.
- Nosek, J. T. 1998. "The Case for Collaborative Programming," *Communications of the ACM* (41:3), pp. 105-108.
- Orr, K. 2002. "CMM Versus Agile Development: Religious Wars and Software Development," *Agile Project Management Executive Report* (3:7), Cutter Consortium, Arlington, MA.
- Paulus, P. B., Dzindolet, M. T., Poletes, G., and Camacho, L. M. 1993. "Perception of Performance in Group Brainstorming: The Illusion of Group Productivity," *Personality & Social Psychology Bulletin* (19:1), pp. 78-89.
- Petty, R. E., Harkins, S. G., Williams, K. D., and Latane, B. 1977. "The Effects of Group Size on Cognitive Effort and Evaluation," *Personality and Social Psychology Bulletin* (3), pp. 575-578.
- Phongpaibul, M., and Boehm, B. 2006. "An Empirical Comparison Between Pair Development and Software Inspection in Thailand," in *Proceedings of the ACM/IEEE International Symposium on Empirical Software Engineering*, Rio de Janeiro, Brazil, pp. 85-94.
- Rostaher, M., and Hericko, M. 2002. "Tracking Test First Pair Programming—An Experiment," in *Extreme Programming and Agile Methods – XP/Agile Universe 2002*, D. Wells and L. Williams (eds.), Heidelberg: Springer-Verlag, Heidelberg, pp. 3-19.
- Rubinstein, D. 2007. "Standish Group Report: There's Less Development Chaos Today," *Software Development Times* (<http://www.sdtimes.com/content/article.aspx?ArticleID=30247>).
- Schroder, H., Driver, M., and Streufert, S. 1967. *Human Information Processing*, New York: Holt, Rinehart and Winston.
- Simmel, G. (ed.). 1950. *The Sociology of George Simmel*, New York: The Free Press.
- Sniezek, J. A. 1992. "Groups under Uncertainty: An Examination of Confidence in Group Decision Making," *Organizational Behavior & Human Decision Processes* (52:1), pp. 124-155.
- Steiner, I. D. 1972. *Group Process and Productivity*, New York: Academic Press.
- Stephenson, G. M., and Wagner, W. 1989. "Origins of the Misplaced Confidence Effect in Collaborative Recall," *Applied Cognitive Psychology* (3:3), pp. 227-236.
- Vanhnen, J., and Lassenius, C. 2005. "Effects of Pair Programming at the Development Team Level: An Experiment," in *Proceedings of the International Symposium on Empirical Software Engineering*, Noosa, Australia, pp. 336-345.
- Williams, K. D., Harkins, S. G., and Latane, B. 1981. "Identifiability as a Deterrent to Social Loafing: Two Cheering Experiments," *Journal of Personality and Social Psychology* (40), pp. 303-311.
- Williams, L. 2000. *The Collaborative Software*, unpublished Ph.D. dissertation, University of Utah.
- Williams, L. A., and Kessler, R. R. 2000. "All I Really Need to Know About Pair Programming I Learned in Kindergarten," *Communications of the ACM* (43:5), pp. 108-114.
- Williams, L. A., Kessler, R. R., Cunningham, W., and Jeffries, R. 2000. "Strengthening the Case for Pair Programming," *IEEE Software* (17:4), pp. 19-25.
- Xu, S., and Rajlich, V. 2006. "Empirical Validation of Test-Driven Pair Programming in Game Development," in *Proceedings of the 5<sup>th</sup> IEEE/ACIS International Conference on Computer and Information Science*, Honolulu, HI, pp. 500-505.
- Zajonc, R. B. 1965. "Social Facilitation," *Science* (149), pp. 269-274.

## About the Authors

**VenuGopal Balijepally** is an assistant professor of MIS in the College of Business at Prairie View A&M University, Texas. He received his Ph.D. in Information Systems from the University of Texas at Arlington and Post Graduate Diploma in Management (MBA), from the Management Development Institute, Gurgaon, India. His research interests include software development, social capital of IS teams, knowledge management and IT management. His research publications appear in *Communications of the ACM*, *Communications of the AIS*, and various conference proceedings such as the Americas Conference on Information Systems, the Hawaii International Conference on System Sciences, and the Decision Sciences Institute.

**RadhaKanta Mahapatra** is an associate professor of Information Systems at the University of Texas at Arlington. He holds a Ph.D. in Information Systems from Texas A&M University. His research interests include software development methodologies, knowledge management, data mining, web-based end-user training, and IT management. His research publications appear in such journals as *MIS Quarterly*, *Communications of the ACM*, *Decision Support Systems*, *Information & Management*, *Database for Advances in Information Systems*, *Communications of the AIS*, *Journal of Database Management*, *Journal of Computer Information Systems*, and *International Journal of Production Research*. He received the Distinguished Research Publication Award and the Distinguished Professional Publication Award from the College of Business Administration of the University of Texas at Arlington.

**Sridhar Nerur** is an associate professor of Information Systems at the University of Texas at Arlington. He received his Ph.D. from the University of Texas at Arlington. His research interests include software development, citation analysis, reuse, maintenance, and philosophical aspects of systems development. He has published in the *MIS Quarterly*, *Strategic Management Journal*, *Communications of the ACM*, *Database for Advances in Information Systems*, *Communications of the AIS*, *Information Systems Management*, *Information Management & Computer Security*, and in various conference proceedings. He is an associate editor of the *European Journal of Information Systems*.

**Kenneth H. Price** is a professor of Management and Organizational Behavior at the University of Texas at Arlington. He received his Ph.D. from Michigan State University in Industrial-Organizational Psychology. His research publications appear in *Management Science*, *Academy of Management Journal*, *Journal of Management*, *Organizational Behavior and Human Decision Processes*, *Journal of Applied Psychology*, *Journal of Information Systems*, *Personality*

and *Social Psychology Bulletin*, *Group and Organizational Studies*, *Basic and Applied Social Psychology*, *Human Resource Management Review*, *Behavioral Research in Accounting*, *The Accounting Review*, and *Theory and Decision*. He is a past associate editor of the *International Journal of Conflict Management* and was a guest editor and is currently a member of the Editorial Board of *Organizational Behavior and Human Decision Processes*.

## Appendix A

### Task of Low Complexity (Student Grades)

You are to complete an application that involves a Student class. An initial effort at creating the Student class resulted in Student.java. Complete the methods in the Student class. A brief description of these methods follows:

- *Student(String id, String name)* – constructor for the Student class. The two parameters are for initializing the student's id and name.
- *getScores()* – A student has two exam scores. This method prompts the user for the score on each of the exams.
- *computeAverage()* – This method computes the average of the two exam scores and returns the value.
- *computeGrade()* – The purpose of this method is to compute a letter grade based on the average of the two exam scores. The grade calculation is
  - average score  $\geq 90.0$  is an "A"
  - average score between 80.0 and 89.99 is a "B"
  - average score between 70.0 and 79.99 is a "C"
  - average score between 60.0 and 69.99 is a "D"
  - average score  $< 60.0$  is an "F"
 Notice that the method returns a character.
- *toString()* – This method is used to convert a student object to a String. It returns a Student record as a string that contains the id, name, average score, and letter grade.

After you implement these methods, complete the StudentTest.java application. The application creates three students objects and adds them to the array. It then gets the exam scores for each of the students. Finally, display the three students in the following format:

| ID | Name | Average | Grade |
|----|------|---------|-------|
|----|------|---------|-------|

Example:

Assume that we have the following student objects:

Student 1: studentID = "111", name = "Doug Walters", examOneScore = 90.0, examTwoScore = 80.0  
 Student 2: studentID = "222", name = "Garry Sobers", examOneScore = 92.0, examTwoScore = 100.0  
 Student 3: studentID = "333", name = "Viv Richards", examOneScore = 80.0, examTwoScore = 78.0

The output is

|     |              |       |   |
|-----|--------------|-------|---|
| 111 | Doug Walters | 85.00 | B |
| 222 | Garry Sobers | 96.00 | A |
| 333 | Viv Richards | 79.00 | C |

## Appendix B

### Task of High Complexity (Movie Rental Application)

*Summary of task: Provide a way to keep a list of movies and a way to display a movie, given its title. Assume that movie titles are unique.*

An incomplete application is available to fulfill this task. The application displays three options: (1) Add Movie; (2) Display Movie; and (3) Exit. These options have not been implemented. It is your responsibility to implement them. A brief description of each option follows:

- *Add Movie:* This option should prompt the user for a title, create a movie object using the title, and then add the movie to a list of movies maintained in another class.
- *Display Movie:* This option asks for a title, retrieves the movie and displays its title. If the given title is not found, it should display a message to that effect.
- *Exit:* Exits the application.

Modify the Movie class to include the following instance variables:

- *Category* – which tells you what type of movie it is. This could be “Comedy,” “Mystery,” “Western,” “Classic,” “Action,” etc.
- *Rating* – indicates the rating of the movie (e.g., PG, PG-13, R, NC-17, etc.)

Write appropriate constructor(s) and methods to handle these changes.

Modify the addMovie method in the user interface to reflect these changes. In other words, the method should now get title, category, and rating, and then create a movie object with those values.

Modify the displayMovie method in the user interface to display title, category, and rating of the retrieved movie.

Add a menu option called *About*. Make it the third option and make *Exit* the fourth option. The *About* option displays a message that reads as follows:

Movie list application programmed by *your name*

# Appendix C

## Software Quality Assessment Rubric for Task of Low Complexity

Session \_\_\_\_\_

Room \_\_\_\_\_

| SNo | Description                                                                                                                                                                                                                                                                                                                                  | Max Points | Points Scored | Remarks |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|---------------|---------|
| I   | <b>Student class evaluation</b>                                                                                                                                                                                                                                                                                                              |            |               |         |
| A   | <b>Student(String id, String name)</b> <ul style="list-style-type: none"> <li>• Proper parameters</li> <li>• Visibility: public</li> <li>• No return type</li> <li>• Method name: same as class name (i.e. Student)</li> <li>• Should only initialize id and name of student</li> <li>• For name, this.name = name should be used</li> </ul> | 10         |               |         |
| B   | <b>getScores()</b> <ul style="list-style-type: none"> <li>• visibility: public</li> <li>• return type: void</li> <li>• assignments</li> </ul>                                                                                                                                                                                                | 20         |               |         |
|     | Give 5 additional points if the getScores() method is generic. That is, it asks the user for the number of courses and then gets that many exam scores.                                                                                                                                                                                      | 5          |               |         |
| C   | <b>computeAverage()</b>                                                                                                                                                                                                                                                                                                                      | 10         |               |         |
| D   | <b>computeGrade()</b>                                                                                                                                                                                                                                                                                                                        | 12         |               |         |
| E   | <b>toString()</b>                                                                                                                                                                                                                                                                                                                            | 10         |               |         |
| II  | <b>Student Test Class Evaluation:</b>                                                                                                                                                                                                                                                                                                        |            |               |         |
| A   | <b>Creating three student objects</b> (to be put in the array called students):                                                                                                                                                                                                                                                              |            |               |         |
| A1  | Option 1:<br>Solution is hard coded.<br>E.g., students[0] = new Student("111", "Viv Richards");                                                                                                                                                                                                                                              | 9          |               |         |
| A2  | Option 2:<br>Generic solution using loop: Example..<br><pre>for(int i = 0; i &lt; students.length; i++) {     //code for initializing }</pre>                                                                                                                                                                                                | 14         |               |         |
| B   | <b>Getting the exam scores for each</b>                                                                                                                                                                                                                                                                                                      |            |               |         |
| B1  | Option 1: No loop                                                                                                                                                                                                                                                                                                                            | 9          |               |         |
| B2  | Option 2: With loop (generalizable):                                                                                                                                                                                                                                                                                                         | 14         |               |         |
| C   | <b>Displaying the three students:</b>                                                                                                                                                                                                                                                                                                        |            |               |         |
| C1  | Option 1: Hard coded solution (no loop, don't use toString() concept etc.): 20 Points<br>Use the following criteria for evaluation: <ul style="list-style-type: none"> <li>• output format</li> <li>• accuracy of results</li> </ul>                                                                                                         | 20         |               |         |
| C2  | Option2: Generic solution. They use a loop and System.out.println(studentObject).                                                                                                                                                                                                                                                            | 25         |               |         |
| III | <b>Going beyond requirements:</b>                                                                                                                                                                                                                                                                                                            |            |               |         |
|     | Maintainability considerations – <ul style="list-style-type: none"> <li>• appropriate comments</li> <li>• indentation</li> </ul>                                                                                                                                                                                                             | 5          |               |         |
|     | <b>Total</b>                                                                                                                                                                                                                                                                                                                                 | <b>125</b> |               |         |

# Appendix D

## Software Quality Assessment Rubric for Task of High Complexity

Session \_\_\_\_\_ Room \_\_\_\_\_

| SNo        | Description                                                                                                                                                                                                                                                                          | Max Points | Points Scored | Remarks |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|---------------|---------|
| <b>I</b>   | <b>Movie class evaluation</b>                                                                                                                                                                                                                                                        |            |               |         |
| A          | Add 2 String variables for <b>category</b> and <b>rating</b><br>If data type is wrong, deduct 3 points                                                                                                                                                                               | 5          |               |         |
| B          | Add a constructor:<br>public Movie(String title, String category, String rating)<br>{<br>this.title = title;<br>this.category = category;<br>this.rating = rating;<br>}<br>Check for: return type (none for constructor), variable names – deduct some points if they are incorrect. | 10         |               |         |
| C          | Add getter/accessor methods for category and rating                                                                                                                                                                                                                                  | 5          |               |         |
| D          | Additional points if toString() is implemented                                                                                                                                                                                                                                       | 5          |               |         |
| <b>II</b>  | <b>Display of menu – if correct</b>                                                                                                                                                                                                                                                  | 5          |               |         |
| <b>III</b> | <b>AddMovie class evaluation</b>                                                                                                                                                                                                                                                     |            |               |         |
| A          | Get title, category, rating (5 points each)                                                                                                                                                                                                                                          | 15         |               |         |
| B          | Create a movie object:                                                                                                                                                                                                                                                               |            |               |         |
| B1         | Make sure they have a reference to MovieCollection in the UserInterface class<br><br><b>OR</b><br>They could handle this by making the add and get methods static in the MovieCollection class                                                                                       | 5          |               |         |
| B2         | Movie object created with proper parameters                                                                                                                                                                                                                                          | 10         |               |         |
| C          | Adding the movie – proper call to MovieCollection's "add" method                                                                                                                                                                                                                     | 5          |               |         |
| <b>IV</b>  | <b>Implementation of Display Movie Option</b>                                                                                                                                                                                                                                        |            |               |         |
| A          | Adding a "get" method to the MovieCollection that takes the title as a parameter and return a movie object                                                                                                                                                                           | 10         |               |         |
| A1         | if they display "Title not found" when movie is not found.                                                                                                                                                                                                                           | 5          |               |         |
| A2         | Additional points if they use exceptions                                                                                                                                                                                                                                             | 10         |               |         |
| B          | Getting/reading the title                                                                                                                                                                                                                                                            | 5          |               |         |
| C          | Displaying the movie                                                                                                                                                                                                                                                                 | 10         |               |         |
| C1         | Additional points if toString() is implicitly used                                                                                                                                                                                                                                   | 5          |               |         |
| <b>V</b>   | <b>"About" option implementation</b>                                                                                                                                                                                                                                                 | 5          |               |         |
| <b>VI</b>  | <b>Exit – if correct</b>                                                                                                                                                                                                                                                             | 5          |               |         |
| <b>VII</b> | <b>Going beyond requirements:</b>                                                                                                                                                                                                                                                    |            |               |         |
|            | Maintainability considerations –<br>• appropriate comments<br>• indentation                                                                                                                                                                                                          | 5          |               |         |
|            | <b>Total</b>                                                                                                                                                                                                                                                                         | <b>125</b> |               |         |

Copyright of MIS Quarterly is the property of MIS Quarterly & The Society for Information Management and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.